SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT – 3

CRYPTOGRAPHY AND NETWORK SECURITY

Course Outcome (CO3): Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation.

Submitted by: D.S. Tharuniya Priya
Register Number: 192511317
Github Repo Link : https://github.com/tharu-ui/CSA5112---Cryptography-and-Network-Security/tree/main/Assignments/Assignment%203

Q1 – WHAT MAKES A HASH FUNCTION TRUSTWORTHY? (6 MARKS)

Part A – Avalanche and Fixed-Length Digests

Shared Group Work

Together:

● Understand the required properties of a cryptographic hash (pre-image, second pre-image and collision resistance, avalanche effect)

● Implement a deliberately weak toy hash (for example, a byte-sum or XOR-fold hash) alongside a standard hash (SHA-256 from a library, used for comparison only)

● Decide how "avalanche" will be measured

● Run and test the experiment

Student A – Design & Mathematical Analysis

Student A must individually:

● Explain what each property means as an attack scenari

● Derive the expected number of output bits that change when one input bit is

flipped in an ideal hash

● Explain mathematically why the toy hash fails the avalanche and second pre-image properties

● Explain why a fixed-length digest must have collisions

Student B – Experimental Validation

Student B must individually:

● Design the experimental inputs (single-bit flips, similar messages, long and short messages)

● Instrument the code to count the number of changed output bits ● Test at least three message sizes and a minimum number of trials per size ● Compare predicted and observed bit-change averages

● Explain any discrepancy

Part B – Collision Effort and the Birthday Bound

Shared Group Work

Together, build a collision finder for a truncated hash (take only the first k bits of SHA-256, for k = 16, 20, 24, 28, 32).

Student A – Experimental Investigation

Student A must:

● Design the collision-search runs and repeat each several times ● Record the number of hashes computed before the first collision ● Investigate increasing values of k

● Present the experimental evidence in a table and graph

Student B – Birthday-Bound Reasoning

Student B must:

● Derive why the expected effort is roughly 2^(k/2), not 2^k

● Explain the difference between finding any collision and finding a collision with a specific message

● Explain how this determines the minimum safe digest length ● Relate the result to the retirement of MD5 and SHA-1

Individual Verification

Faculty may ask Student A to explain the birthday bound and Student B to explain the avalanche experiment. Primary responsibility does not mean exclusive knowledge.

1.1 Introduction

A cryptographic hash function is an algorithm that converts an input message of arbitrary length into a fixed-length output called a hash value or message digest.

Hash functions are widely used in password storage, digital signatures, file integrity verification, blockchain technology and authentication systems.

A secure cryptographic hash function must provide pre-image resistance, second pre-image resistance, collision resistance and the avalanche effect.

1.2 Properties of a Cryptographic Hash Function

1. Pre-image Resistance

Given a hash value H(m), it should be computationally infeasible to determine the original message m.

For an ideal n-bit hash function, an attacker may need approximately 2^n attempts to find a message corresponding to a given digest.

2. Second Pre-image Resistance

Given an existing message m1, it should be computationally infeasible to find another message m2 such that:

H(m1) = H(m2)

This prevents an attacker from replacing an existing message with a different message having the same digest.

3. Collision Resistance

It should be computationally infeasible to find any two different messages m1 and m2 such that:

H(m1) = H(m2)

Unlike a second pre-image attack, the attacker is free to choose both messages.

4. Avalanche Effect

A small change in the input should produce a significant and unpredictable change in the output.

Ideally, changing one input bit should change approximately half of the output bits.

1.3 Mathematical Analysis of the Avalanche Effect

SHA-256 produces a digest of 256 bits.

If the hash function behaves ideally, each output bit has approximately a 50% probability of changing when one input bit is modified.

Therefore:

Expected changed bits = 256 × 0.5

Expected changed bits = 128 bits

This means that changing only one bit in the input should change approximately 128 bits in the SHA-256 output.

The number of changed bits is measured using the Hamming distance between the original digest and the modified digest.

Hamming distance = Number of bit positions having different values.

A larger and unpredictable Hamming distance indicates a stronger avalanche effect.

1.4 Why the Toy Hash Functions Are Weak

Two deliberately weak hash functions were implemented for comparison: Toy Sum and Toy XOR.

Toy Sum Hash

The Toy Sum hash adds the byte values of the input message.

Hash = Sum of all input byte values

It does not distribute changes throughout the output. A small modification may affect only a limited portion of the result.

For example, two messages can have the same sum even though their contents and byte order are different.

Toy XOR Hash

The Toy XOR hash combines the bytes using the XOR operation.

Hash = Byte1 XOR Byte2 XOR ... XOR ByteN

XOR has properties that make this construction weak. For example, identical values cancel each other:

A XOR A = 0

Consequently, different messages can easily produce the same result. The output also does not change sufficiently when an input bit is modified.

Neither construction provides the security properties expected from a cryptographic hash function.

1.5 Experimental Procedure

A Python program named Q1_Hash_Experiment.py was implemented to compare Toy Sum, Toy XOR and SHA-256.

The experiment was conducted using messages of three different sizes:

- 16 bytes
- 64 bytes
- 256 bytes

For each message size, 100 trials were performed. One input bit was modified, and the number of changed output bits was calculated.

The experiment also implemented a collision finder using the first k bits of SHA-256, where k was varied from 16 to 32.

1.6 Experimental Results – Avalanche Effect

Hash Function	Message Size (Bytes)	Trials	Average Changed Bits
Toy Sum	16	100	2.00
Toy Sum	64	100	1.89
Toy Sum	256	100	2.16
Toy XOR	16	100	1.00
Toy XOR	64	100	1.00
Toy XOR	256	100	1.00
SHA-256	16	100	126.67
SHA-256	64	100	127.26
SHA-256	256	100	128.01

1.7 Analysis of Results

The expected number of changed bits for SHA-256 is 128.

The experimental results for SHA-256 were approximately 127 to 128 bits for all three message sizes. These values are close to the theoretical expectation.

Toy Sum produced an average of approximately 2 changed bits, while Toy XOR produced only 1 changed bit.

This demonstrates that the toy hash functions do not provide a strong avalanche effect.

The small variations in SHA-256 measurements are expected because the experiment uses a finite number of trials. A cryptographic hash does not guarantee exactly 128 changed bits for every individual input.

1.8 Birthday Bound and Collision Resistance

A hash function with a k-bit output has 2^k possible digest values.

If an attacker searches for any two messages with the same digest, the probability of a collision increases much faster than a simple search for one specific digest.

This is explained by the Birthday Paradox.

The approximate effort required to find a collision is:

Expected collision effort ≈ 2^(k/2)

The reason is that the number of possible pairs among N generated messages is approximately N(N − 1)/2. A collision becomes likely when the number of pairs becomes comparable to the number of possible digest values.

The approximate 50% collision probability threshold is:

N ≈ 1.177 × 2^(k/2)

The program uses 2^(k/2) as an approximate birthday-bound comparison value.

Experimental Results – Collision Search

The collision finder was executed three times for each digest length.

Digest Length (k bits)	Trial 1	Trial 2	Trial 3	Average Hashes	Approx. Birthday Bound
16	304	759	97	386.67	256
20	1451	1716	1809	1658.67	1024
24	4897	3732	4186	4271.67	4096
28	19132	10733	18358	16074.33	16384
32	39086	94112	79904	71034.00	65536

1.9 Analysis of Collision Results

The results show that the number of hashes required to find a collision generally increases as the digest length increases.

For a 16-bit digest, collisions were found after relatively few attempts. As the digest length increased to 32 bits, the average number of attempts increased significantly.

The experimental values do not exactly match the theoretical estimates because collision searches are probabilistic. The first collision may occur earlier or later depending on the generated messages and their digest values.

Difference Between Finding Any Collision and Finding a Specific Collision

- Finding any collision means identifying any two different messages with the same digest. The approximate effort is 2^(k/2).
- Finding a second pre-image means finding a different message that matches the digest of one specific existing message. For an ideal k-bit hash, the approximate effort is 2^k.

Therefore, finding any collision is generally easier than finding a second pre-image.

1.10 Importance of Digest Length and Algorithm Choice

A longer digest provides a larger output space and increases the computational effort required for collision attacks.

However, digest length alone does not guarantee security. The algorithm must also be resistant to known attacks.

MD5 and SHA-1 are no longer recommended for security-sensitive digital signatures because practical collision attacks have been demonstrated against them.

SHA-256 and SHA-3 are suitable modern choices for many cryptographic applications.

For an ideal 256-bit hash, the approximate generic collision effort is 2^128 operations. This is far beyond practical brute-force capability using currently available computing resources.

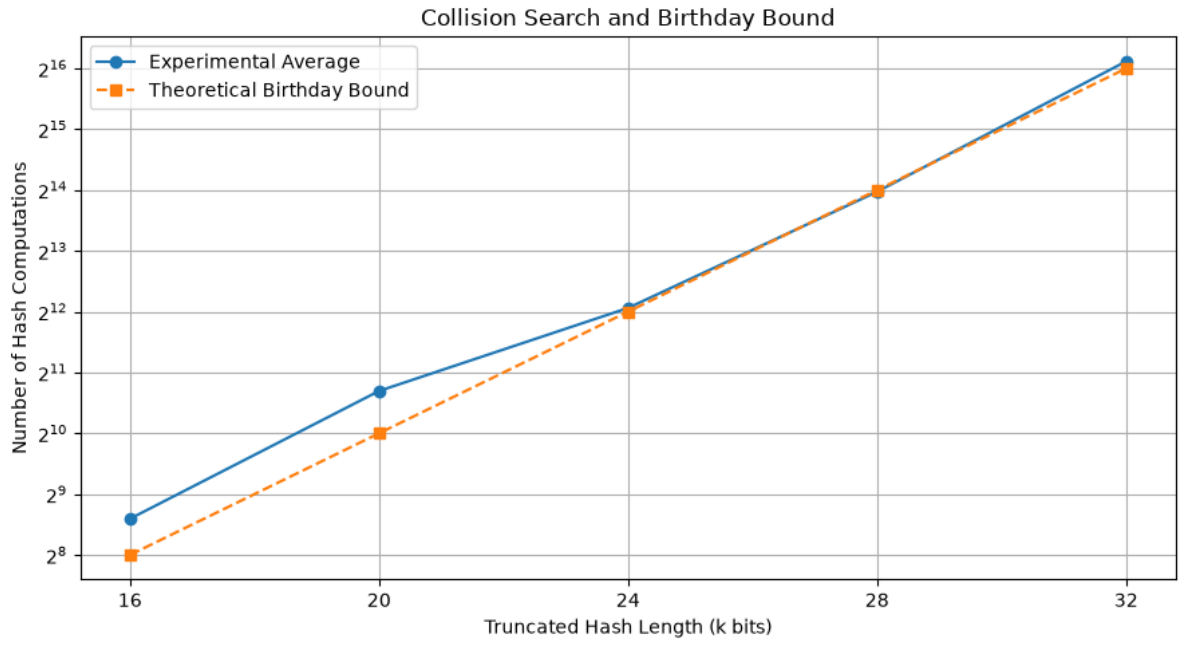
1.11 Conclusion

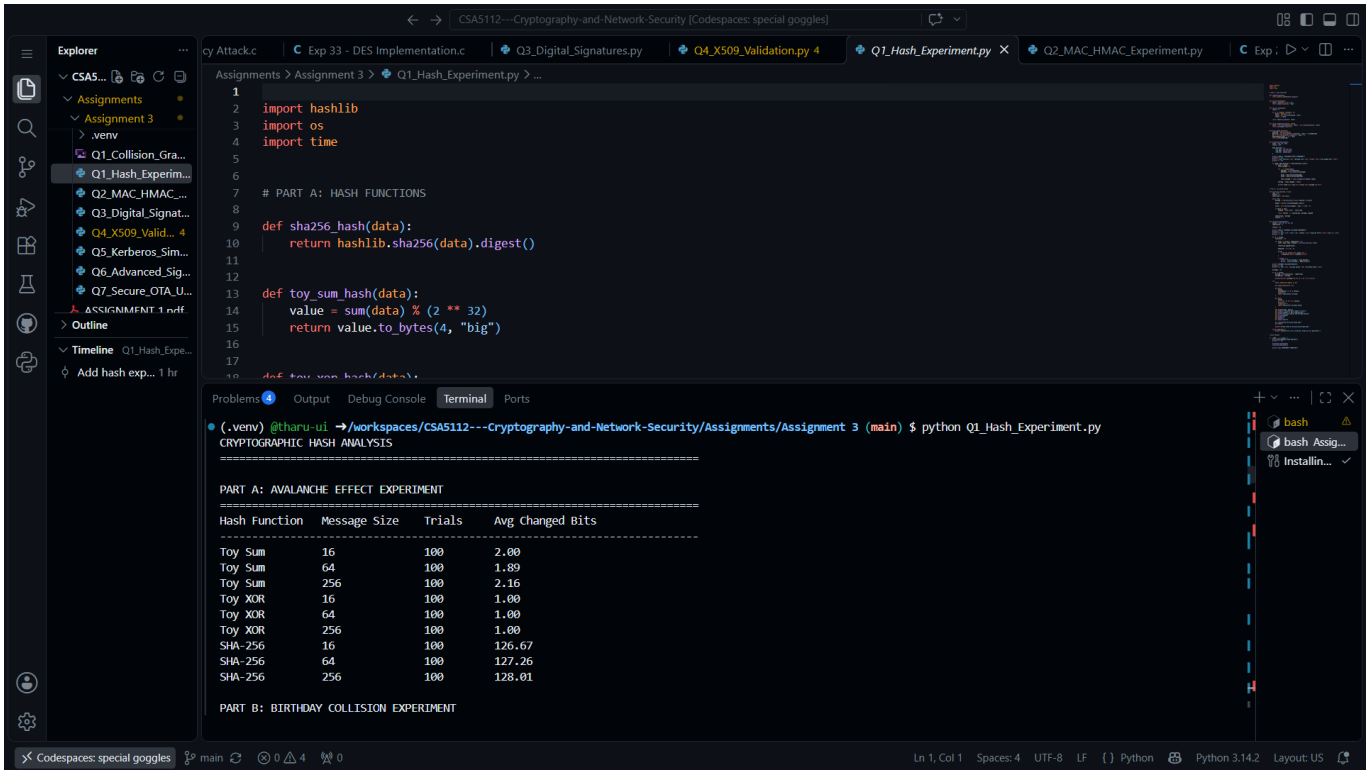
The experiment demonstrated the importance of avalanche effect, collision resistance and digest length in cryptographic hash functions.

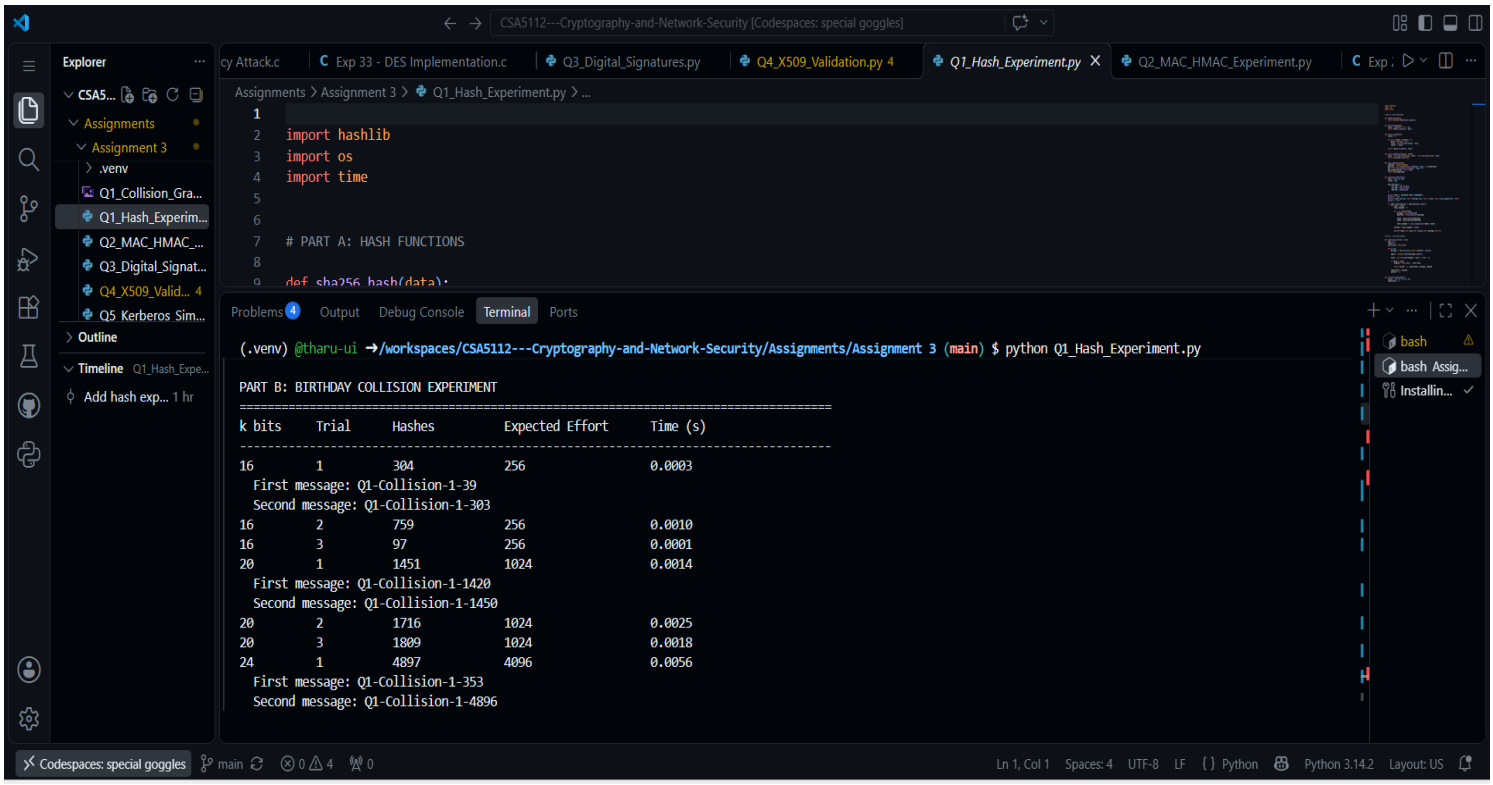
Toy Sum and Toy XOR showed weak avalanche behaviour, whereas SHA-256 produced approximately 128 changed bits out of 256.

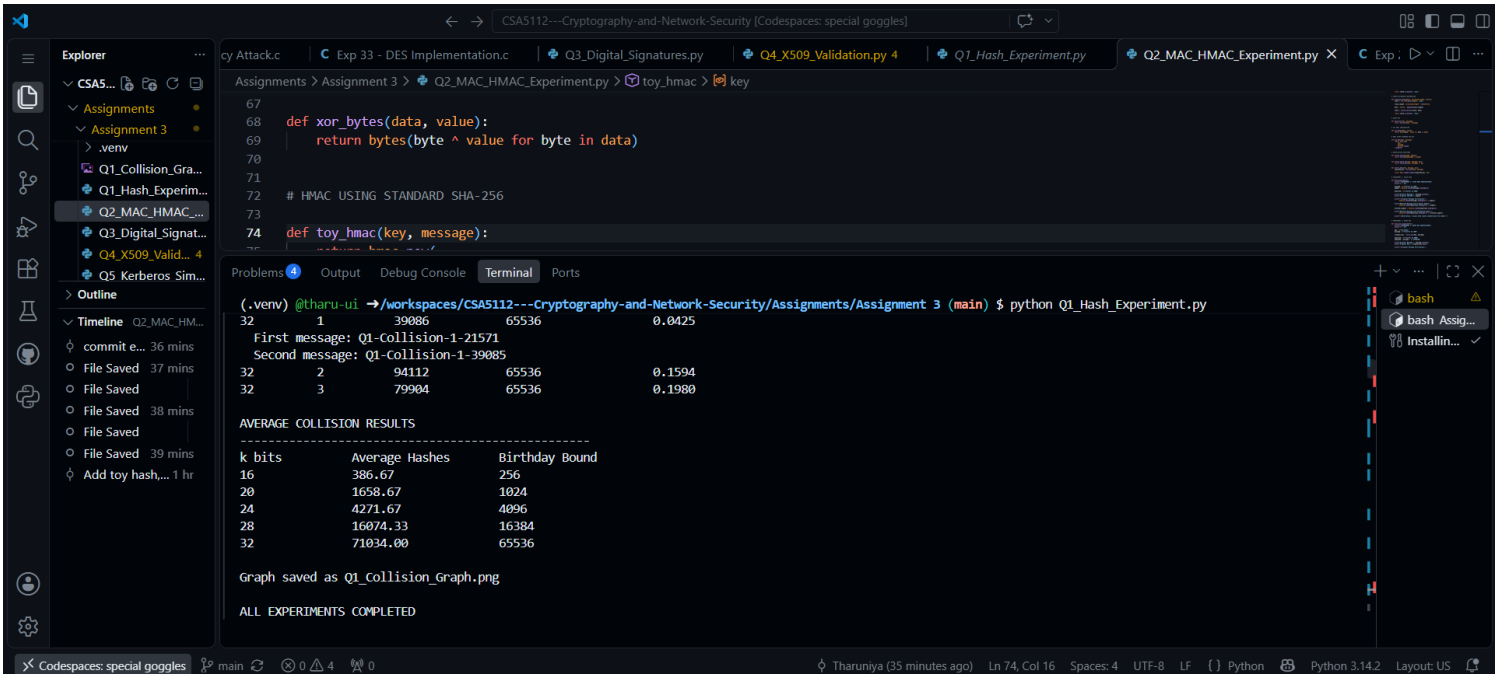
The collision experiment also demonstrated that the effort required to find collisions increases with digest length and follows the birthday-bound principle.

Therefore, secure applications should use established cryptographic hash algorithms such as SHA-256 or SHA-3 rather than simple custom hash constructions.




Explorer
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Valid... 4
Q5_Kerberos_Sim...
Q6_Advanced_Sig...
Q7_Secure_OTA_U...
Outline
Timeline
Add hash exp... 1 hr
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q1_Hash_Experiment.py > ...
import hashlib
import os
import time
PART A: HASH FUNCTIONS
def sha256_hash(data):
return hashlib.sha256(data).digest()
def toy_sum_hash(data):
value = sum(data) % (2 ** 32)
return value.to_bytes(4, "big")
Problems 4
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q1_Hash_Experiment.py
CRYPTOGRAPHIC HASH ANALYSIS
PART A: AVALANCHE EFFECT EXPERIMENT
Hash Function Message Size Trials Avg Changed Bits
Toy Sum 16 100 2.00
Toy Sum 64 100 1.89
Toy Sum 256 100 2.16
Toy XOR 16 100 1.00
Toy XOR 64 100 1.00
Toy XOR 256 100 1.00
SHA-256 16 100 126.67
SHA-256 64 100 127.26
SHA-256 256 100 128.01
PART B: BIRTHDAY COLLISION EXPERIMENT
bash
bash Assig...
Installin...
Codespaces: special goggles
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US


Explorer
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Valid... 4
Q5_Kerberos_Sim...
Outline
Timeline
Add hash exp... 1 hr
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q1_Hash_Experiment.py > ...
import hashlib
import os
import time
PART A: HASH FUNCTIONS
Problems 4
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q1_Hash_Experiment.py
PART B: BIRTHDAY COLLISION EXPERIMENT
k bits Trial Hashes Expected Effort Time (s)
16 1 304 256 0.0003
First message: Q1-Collision-1-39
Second message: Q1-Collision-1-303
16 2 759 256 0.0010
16 3 97 256 0.0001
20 1 1451 1024 0.0014
First message: Q1-Collision-1-1420
Second message: Q1-Collision-1-1450
20 2 1716 1024 0.0025
20 3 1809 1024 0.0018
24 1 4897 4096 0.0056
First message: Q1-Collision-1-353
Second message: Q1-Collision-1-4896
bash
bash Assig...
Installin...
Codespaces: special goggles
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US


CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Valid... 4
Q5 Kerberos Sim...
Outline
Timeline Q2_MAC_HM...
commit e... 36 mins
File Saved 37 mins
File Saved
File Saved 38 mins
File Saved
File Saved 39 mins
Add toy hash,... 1 hr
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py 4
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
67
68 def xor_bytes(data, value):
69 return bytes(byte ^ value for byte in data)
70
71
72 # HMAC USING STANDARD SHA-256
73
74 def toy_hmac(key, message):
Problems 4
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui →/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q1_Hash_Experiment.py
32 1 39086 65536 0.0425
First message: Q1-Collision-1-21571
Second message: Q1-Collision-1-39085
32 2 94112 65536 0.1594
32 3 79904 65536 0.1980
AVERAGE COLLISION RESULTS
k bits Average Hashes Birthday Bound
16 386.67 256
20 1658.67 1024
24 4271.67 4096
28 16074.33 16384
32 71034.00 65536
Graph saved as Q1_Collision_Graph.png
ALL EXPERIMENTS COMPLETED
bash
bash Assig...
Installin...
Codespaces: special goggles
main
Tharuniya (35 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US

Q2 – INTEGRITY IS NOT THE SAME AS AUTHENTICATION: HASH, MAC AND HMAC (7 MARKS)

Problem Context

A software distributor publishes a file together with its SHA-256 digest on the same web page. Another company protects internal messages with the construction H(key || message).

Shared Group Work

The pair must:

● Implement a simple Merkle–Damgård style toy hash (fixed block size, chaining value, padding with length)

● Implement the naive MAC H(key || message) and the HMAC construction ● Design tamper scenarios (modified message, modified message and recomputed digest, appended data)

● Conduct the experiments

Student A – Attack and Construction Investigation

Student A is individually responsible for:

● Showing that an attacker who can change both the file and the published digest defeats a plain hash

● Demonstrating a length-extension attack on the naive H(key || message) using your toy Merkle–Damgård hash

● Recording what the attacker needs to know and what the attacker does not need to know

● Explaining which property of the chaining construction makes the attack possible

Student B – Comparative Security Analysis

Student B is individually responsible for:

•Explaining the structure of HMAC (inner and outer hash, key padding) and why it resists length extension

● Comparing hash-only, naive MAC and HMAC in a table: which provides integrity, which provides authentication, and which provides neither ● Explaining why a MAC cannot provide non-repudiation

● Verifying through tests that HMAC detects every tamper scenario you designed

Joint Design Question

Together answer:

If you were designing the download-verification and message-protection components for an organisation, which mechanism would you choose for each, what must be kept secret, and what residual risk remains?

2.1 Introduction

Cryptographic hash functions, Message Authentication Codes (MACs) and Hash-based Message Authentication Codes (HMACs) are used to protect data.

Although these mechanisms are related, they provide different security properties.

A plain hash can help detect accidental or unauthorized changes when the expected digest is obtained from a trusted source. However, it does not authenticate the sender.

A MAC uses a secret key to provide message integrity and authentication between parties sharing that key.

HMAC is a standardized and secure construction that uses a cryptographic hash function together with a secret key.

2.2 Plain Hash Verification

A plain hash produces a digest from a message.

Digest = SHA-256(Message)

The sender publishes the message and its digest. The receiver calculates the digest again and compares it with the published value.

If both values match, the receiver knows that the received message corresponds to the published digest.

However, if an attacker can modify both the message and the published digest, the attacker can calculate a new digest for the modified message.

Therefore, publishing a hash beside a file on the same untrusted webpage does not provide reliable sender authentication.

Experimental Procedure

The program Q2_MAC_HMAC_Experiment.py used the message:

Original Message: Transfer Rs.1000

The original SHA-256 digest was calculated. The message was then modified, and verification was performed using both the original and recomputed digests.

Experimental Results

Test Case	**Result**
Original message with original digest	True
Modified message with original digest	False
Modified message with recomputed digest	True

Original SHA-256 digest:

f474ae266e4d8985d8a5518748496230b70beb75f37b6f6ded170dd0eb30f55f

Observation

The original message was successfully verified.

When the message was modified without changing the original digest, verification failed.

However, when the attacker calculated a new digest for the modified message, verification succeeded.

This demonstrates that a plain hash alone cannot establish the authenticity of the sender.

2.3 Merkle–Damgård Construction

The Merkle–Damgård construction is a method used to build hash functions by processing a message in fixed-size blocks.

The message is divided into blocks. Padding is added to the final block, usually including the message length. Each block updates an internal chaining value.

The simplified process is:

1. Divide the message into fixed-size blocks.
2. Add padding and message-length information.
3. Initialize the chaining value.
4. Process each block using a compression function.
5. Produce the final hash value.

The chaining structure is important because the final digest can expose the internal state needed to continue processing additional data in certain poorly designed constructions.

This property contributes to the length-extension vulnerability of a naive MAC.

2.4 Naive MAC: H(key || message)

A naive MAC is constructed by placing a secret key before the message and hashing the combined value.

MAC = H(key || message)

Here:

- H represents the hash function.
- key is a secret value shared between the sender and receiver.
- message is the data being protected.
- || represents concatenation.

Unlike a plain hash, a naive MAC uses a secret key. An attacker who does not know the key cannot normally generate a valid MAC for an arbitrary message.

However, the construction is vulnerable to a length-extension attack when used with a vulnerable Merkle–Damgård hash.

Experimental Results – Naive MAC

Original Message: Transfer Rs.1000

Original MAC: 91af678c

Test Case	Verification Result
Original message	True
Modified message	False
Appended message	False

The original message was accepted, while the modified and appended messages were rejected during ordinary verification.

2.5 Length-Extension Attack

A length-extension attack exploits the internal structure of certain Merkle–Damgård hash functions.

When a system uses the construction H(key || message), the final digest can act as the internal chaining state for processing additional data.

An attacker who knows the original message, its MAC and an estimate of the secret key length may be able to calculate a valid MAC for an extended message without knowing the secret key.

Attack Procedure

1. The attacker observes the original message and its MAC.
2. The attacker estimates the length of the secret key.
3. The attacker calculates the padding that would follow the original key and message.
4. The attacker continues the hash computation using the known digest as the chaining state.
5. The attacker generates a forged MAC for the extended message.

6. The server verifies the forged message using the same vulnerable construction.

Experimental Results

Parameter	**Result**
Original Message	Transfer Rs.1000
Original MAC	91af678c
Appended Data	&admin=true
Guessed Key Length	9
Attacker Knows Secret Key	No
Forged MAC	730cd30c
Forged Message Accepted by Server	True

Observation

The experiment successfully demonstrated that the attacker could append additional data and produce a valid MAC without knowing the secret key.

The vulnerability exists because the final digest of the Merkle–Damgård construction can be used as a continuation state.

The attack does not mean that every MAC construction is vulnerable. It specifically demonstrates the weakness of the naive H(key || message) construction with a vulnerable hash design.

2.6 HMAC Construction

HMAC stands for Hash-based Message Authentication Code.

It is a standardized construction that combines a cryptographic hash function with a secret key.

HMAC uses two levels of hashing: an inner hash and an outer hash.

The general formula is:

HMAC(K, M) = H((K' XOR opad) || H((K' XOR ipad) || M))

Where:

- K is the secret key.
- M is the message.
- K' is the key adjusted to the hash function's block size.
- ipad is the inner padding value.
- opad is the outer padding value.
- XOR is the exclusive OR operation.
- H is the selected cryptographic hash function.

Working of HMAC

1. The secret key is adjusted to the required block size.
2. The key is XORed with the inner padding value.
3. The result is combined with the message and hashed.
4. The key is XORed with the outer padding value.
5. The outer key value is combined with the inner hash result.
6. A final hash operation produces the HMAC.

The nested structure prevents an attacker from simply continuing the inner hash computation using the final HMAC value.

In the experiment, the standard HMAC-SHA-256 construction was implemented using Python's hmac and hashlib modules. The verification used hmac.compare_digest().

2.7 Experimental Results – HMAC-SHA-256

Original Message: Transfer Rs.1000

Original HMAC-SHA-256:

2fccadb5980b0fb12da4643f8800bf615b04b2fb972c067e125faebcf7747fec

Test Case	Verification Result
Original message with original HMAC	True
Modified message with original HMAC	False
Appended message with original HMAC	False
Original message with modified HMAC	False
Modified message with newly generated HMAC	True

Observation

HMAC successfully verified the original message and rejected the modified message when the original authentication tag was used.

It also rejected appended data and a modified HMAC value.

The modified message was accepted only when a new valid HMAC was generated using the secret key.

This demonstrates that an attacker without the secret key cannot independently create a valid authentication tag.

2.8 Comparison of Hash, Naive MAC and HMAC

Feature	Plain Hash	Naive MAC	HMAC
Secret key required	No	Yes	Yes
Message integrity	Limited	Yes	Yes
Sender authentication	No	Yes, between key holders	Yes, between key holders
Length-extension resistance	Not applicable as authentication mechanism	Vulnerable in this construction	Resistant
Non-repudiation	No	No	No
Security suitability	Depends on trusted digest source	Not recommended	Recommended

2.9 Why a MAC Cannot Provide Non-Repudiation

A MAC uses a shared secret key known to both the sender and the receiver.

Since both parties possess the same key, either party can generate a valid MAC.

Therefore, if a dispute occurs, the receiver cannot prove to an independent third party that only the sender generated the message.

A digital signature solves this problem differently because the signer uses a private key and verification is performed using the corresponding public key.

2.10 Recommended Mechanisms for an Organisation

For download verification, a plain SHA-256 digest is useful only when the digest itself is obtained through a trusted and authenticated channel. For stronger assurance about the publisher, the organisation should use a digital signature over the file or its digest.

For internal message protection, HMAC-SHA-256 should be used because it provides integrity and authentication to parties sharing the secret key.

The secret key must be securely generated, stored and rotated. It must never be included in publicly accessible files or transmitted without protection.

Residual risks include key compromise, weak key management, insecure software implementations and attacks against systems that do not authenticate the expected digest or signing key.

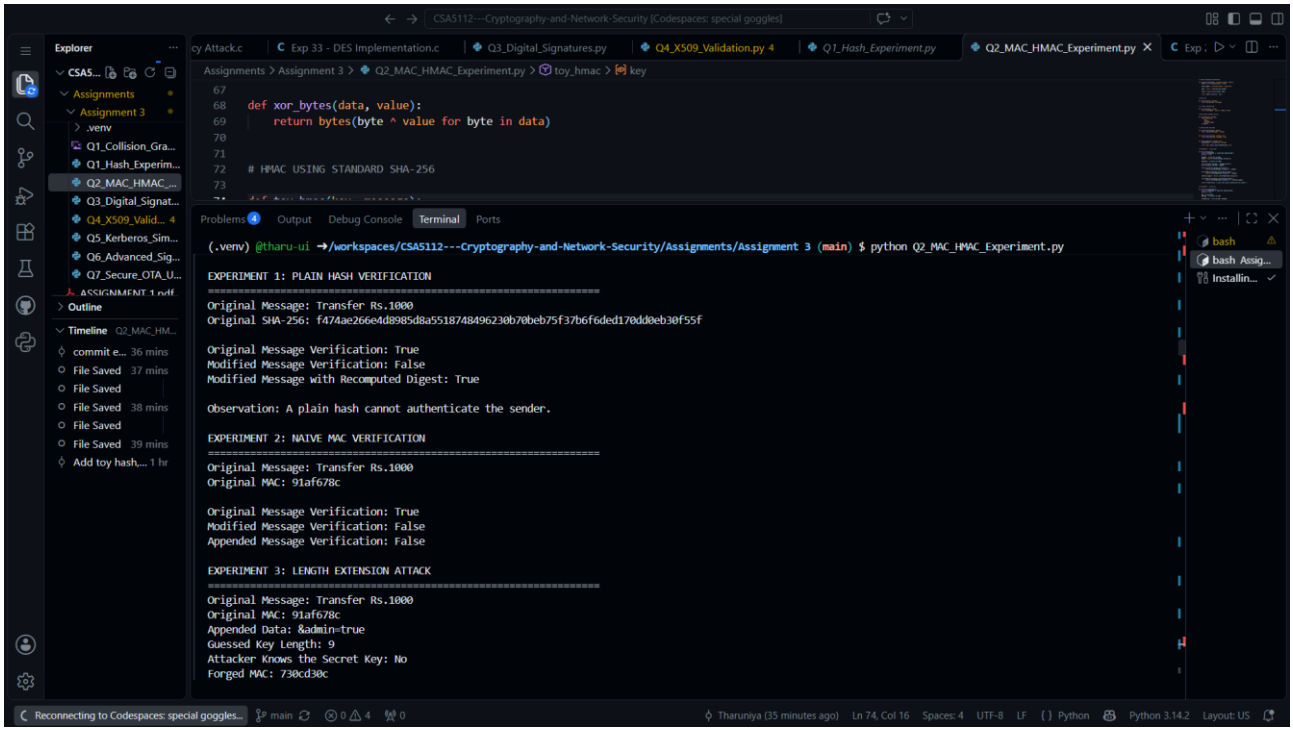
2.11 Conclusion

The experiment demonstrated the difference between integrity and authentication.

A plain hash can detect changes only when the expected digest is trusted. The naive MAC construction was shown to be vulnerable to a length-extension attack.

HMAC-SHA-256 successfully detected the tested tampering attempts and provided message authentication.

Therefore, HMAC is the preferred mechanism for protecting internal messages, while digital signatures are appropriate when public verification and non-repudiation are required.


Explorer
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Valid... 4
Q5_Kerberos_Sim...
Q6_Advanced_Sig...
Q7_Secure_OTA_U...
Outline
Timeline
commit e... 36 mins
File Saved 37 mins
File Saved
File Saved 38 mins
File Saved
File Saved 39 mins
Add toy hash,... 1 hr
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py 4
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
67
68 def xor_bytes(data, value):
69 return bytes(byte ^ value for byte in data)
70
71
72 # HMAC USING STANDARD SHA-256
73
Problems 4
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q2_MAC_HMAC_Experiment.py
EXPERIMENT 1: PLAIN HASH VERIFICATION
Original Message: Transfer Rs.1000
Original SHA-256: f474ae266e4d8985d8a551874849623 0b70beb75f37b6f6ded170dd0eb30f55f
Original Message Verification: True
Modified Message Verification: False
Modified Message with Recomputed Digest: True
Observation: A plain hash cannot authenticate the sender.
EXPERIMENT 2: NAIVE MAC VERIFICATION
Original Message: Transfer Rs.1000
Original MAC: 91af678c
Original Message Verification: True
Modified Message Verification: False
Appended Message Verification: False
EXPERIMENT 3: LENGTH EXTENSION ATTACK
Original Message: Transfer Rs.1000
Original MAC: 91af678c
Appended Data: &admin=true
Guessed Key Length: 9
Attacker Knows the Secret Key: No
Forged MAC: 730cd30c
bash
bash Assig...
Installin...
Reconnecting to Codespaces: special goggles...
main
Tharuniya (35 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US

Q3 – DIGITAL SIGNATURES: HASH-THEN-SIGN AND WHAT CAN BREAK THEM (7 MARKS)

Shared Group Work

Together:

● Implement RSA signature generation and verification using hash-then-sign (small keys for measurement)

● Implement a DSA-style (or ECDSA-style) signature using toy parameters ● Test valid signatures, modified messages, modified signatures and wrong public keys

● Conduct the nonce experiment described below

Student A – Algorithmic Derivation

Student A must individually:

● Explain why the message is hashed before signing and what would go wrong with signing the raw message

● Prove why verification succeeds for a correctly generated RSA signature (s = H(m)^d mod n and s^e ≡ H(m) mod n)

● Derive the DSA/ECDSA verification equation and show why it recovers the signer's value

● Explain what security goal each step contributes (integrity, authentication, non-repudiation)

Student B – Attack Experiment and Evidence

Student B must individually:

● Design the test messages and tamper cases and record accept/reject results ● Demonstrate nonce reuse in DSA/ECDSA: sign two messages with the same per-signature secret k, then recover k and the private key from the two signatures

● Demonstrate a textbook (no hash) RSA signature forgery (for example,

multiplicative property: signature of $m_1 \cdot m_2$ from signatures of m_1 and m_2) ● Explain what these results show about "correct maths but unsafe implementation"

Joint Conclusion

Together answer:

A valid signature proves that someone holding the private key signed the message. Under what conditions does this actually amount to non-repudiation, and what could the signer claim to deny it?

Consider key compromise, timestamping, and the role of a trusted third party.

3.1 Introduction

A digital signature is a cryptographic mechanism used to verify the authenticity and integrity of a digital message.

It uses a pair of keys:

- Private key: Used by the signer to generate the signature.
- Public key: Used by the receiver to verify the signature.

Digital signatures support authentication, integrity and, under suitable conditions, non-repudiation.

RSA and DSA are examples of digital signature algorithms.

3.2 Why Is Hashing Performed Before Signing?

Signing an entire message directly can be inefficient, especially when the message is large.

Therefore, the message is first passed through a cryptographic hash function to produce a fixed-length digest.

The signer then signs the digest using the private key.

The process is:

Message → Hash Function → Message Digest → Private-Key Operation → Digital Signature

The receiver calculates the hash of the received message and verifies the signature using the signer's public key.

If the calculated digest matches the digest authenticated by the signature, the message is considered valid.

Hash-then-sign provides:

- Integrity: Detects modifications to the message.
- Authentication: Connects the signature to the corresponding public key.
- Efficient signing: A fixed-length digest is processed instead of the entire message.

In real systems, secure signature padding and encoding schemes must be used. Raw textbook RSA is not sufficient.

3.3 RSA Digital Signature

RSA is an asymmetric cryptographic algorithm based on the mathematical difficulty of factoring large composite integers.

Key Generation

1. Select two large prime numbers p and q.
2. Calculate n = p × q.
3. Calculate φ(n) = (p − 1) × (q − 1).
4. Select a public exponent e.
5. Calculate the private exponent d such that:

$e \times d \equiv 1 \bmod \varphi(n)$

The public key is (e, n), and the private key is (d, n).

Signature Generation

First calculate the message digest:

h = H(m)

Then generate the signature:

s = h^d mod n

Signature Verification

The receiver calculates:

v = s^e mod n

The signature is valid if:

$v = H(m) \bmod n$

Proof of RSA Verification

The signature is generated using:

$s = H(m)^d \bmod n$

The receiver raises the signature to the public exponent:

$s^e = (H(m)^d)^e \bmod n$

Therefore:

$s^e = H(m)^{(de)} \bmod n$

Since $e \times d \equiv 1 \bmod \varphi(n)$, RSA modular arithmetic gives:

$H(m)^{(de)} \equiv H(m) \bmod n$

Hence:

$s^e \bmod n = H(m) \bmod n$

This proves why a correctly generated RSA signature passes verification.

3.4 DSA-Style Digital Signature

The Digital Signature Algorithm (DSA) is based on the discrete logarithm problem.

The algorithm uses public parameters p, q and g, a private key x and a public key y.

The public key is calculated as:

$y = g^x \bmod p$

A fresh secret nonce k is selected for every signature.

The signature contains two values, r and s.

The signing equations are:

$r = (g^k \bmod p) \bmod q$

$s = k^{-1} \times (H(m) + x \times r) \bmod q$

The receiver calculates:

$w = s^{-1} \bmod q$

$u1 = H(m) \times w \bmod q$

$u2 = r \times w \bmod q$

$v = ((g^{u1} \times y^{u2}) \bmod p) \bmod q$

The signature is valid when $v = r$.

The nonce k must remain secret and must never be reused.

3.5 Experimental Procedure

A Python program named Q3_Digital_Signatures.py was implemented.

The program tested:

1. RSA signature generation and verification.
2. DSA-style signature generation and verification.
3. Nonce reuse in DSA.
4. Textbook RSA multiplicative signature forgery.

The experiments used small toy parameters for educational demonstration.

3.6 Experimental Results – RSA

Parameter	**Result**
Original Message	Secure Communication
Public Key	(65537, 1022117)
Private Key	(832193, 1022117)
Digital Signature	960942

Test Case	**Verification Result**
Original message and signature	True

Modified message	False
Modified signature	False
Wrong public key	False

Observation

The original signature was successfully verified.

When the message, signature or public key was modified, verification failed.

This demonstrates the ability of digital signatures to detect message tampering and incorrect verification keys.

3.7 Experimental Results – DSA-Style Signature

Parameter	**Result**
Prime p	23
Prime q	11
Generator g	2
Private Key	3
Public Key	8
Message	Secure Message
Signature (r, s)	(2, 10)

Test Case	**Verification Result**
Original message and signature	True
Modified message	False
Modified signature	False
Wrong public key	False

The experiment demonstrated that the original DSA-style signature passed verification, whereas the modified message, signature and wrong public key were rejected.

3.8 Nonce Reuse Attack in DSA

A nonce is a secret value used during digital signature generation.

DSA requires a fresh, unpredictable nonce for every signature.

If the same nonce is used to sign two different messages, the private key may be recovered.

For two signatures using the same nonce:

$s1 = k^{-1} \times (h1 + x \times r) \bmod q$

$s2 = k^{-1} \times (h2 + x \times r) \bmod q$

Subtracting the equations gives:

$s1 - s2 = k^{-1} \times (h1 - h2) \bmod q$

Therefore:

$k = (h1 - h2) \times (s1 - s2)^{-1} \bmod q$

After recovering k:

$x = (s1 \times k - h1) \times r^{-1} \bmod q$

Here, x is the private key.

Experimental Results

Parameter	**Result**
Message 1	Transaction Amount 100
Message 2	Transaction Amount 200
Reused Nonce	3
Signature 1	(8, 3)
Signature 2	(8, 5)
Same Nonce Used	True
Same r Value	True
Recovered Nonce	3
Original Private Key	3

Recovered Private Key	3
Private Key Successfully Recovered	True

Observation

The experiment successfully recovered the original private key by using two signatures generated with the same nonce.

This demonstrates that even when the mathematical algorithm is correct, poor randomness management can completely compromise the security of a digital signature system.

3.9 Textbook RSA Multiplicative Signature Forgery

Textbook RSA has a multiplicative property.

If:

$s1 = m1^d \bmod n$

and

$s2 = m2^d \bmod n$

then:

$s1 \times s2 \bmod n = (m1 \times m2)^d \bmod n$

Therefore, a signature for the product of two messages can be constructed by multiplying their signatures.

Experimental Results

Parameter	**Result**
Message 1	5
Message 2	7
Signature 1	808890
Signature 2	78291

Forged Message	35
Forged Signature	481904
Verification Result	35
Textbook RSA Forgery Successful	True

Observation

The experiment successfully generated a signature for the product of two messages without directly signing the combined message.

This demonstrates why textbook RSA signatures are insecure.

Modern RSA signature schemes use secure encoding and padding mechanisms, such as RSA-PSS, to prevent such algebraic forgery attacks.

3.10 Security Goals of Digital Signatures

Security Goal	**Contribution**
Integrity	Any change to the message produces a different digest and invalidates the signature.
Authentication	Verification connects the signature to the public key associated with the signer.
Non-repudiation	Can provide evidence that the holder of a private key signed the message, subject to legal and operational conditions.

3.11 Conditions Required for Non-Repudiation

A valid digital signature alone does not automatically guarantee non-repudiation.

The following conditions are important:

1. The private key must remain under the exclusive control of the signer.
2. The public key must be reliably associated with the signer's identity.
3. The private key must not have been compromised.

4. The signing time should be established through reliable timestamping when required.
5. Certificate validity and revocation status must be checked.
6. Reliable audit records and trusted verification procedures should be maintained.

A signer may dispute a signature by claiming that the private key was stolen, compromised or used without authorization.

A trusted timestamping authority can provide evidence that a signature existed at a particular time.

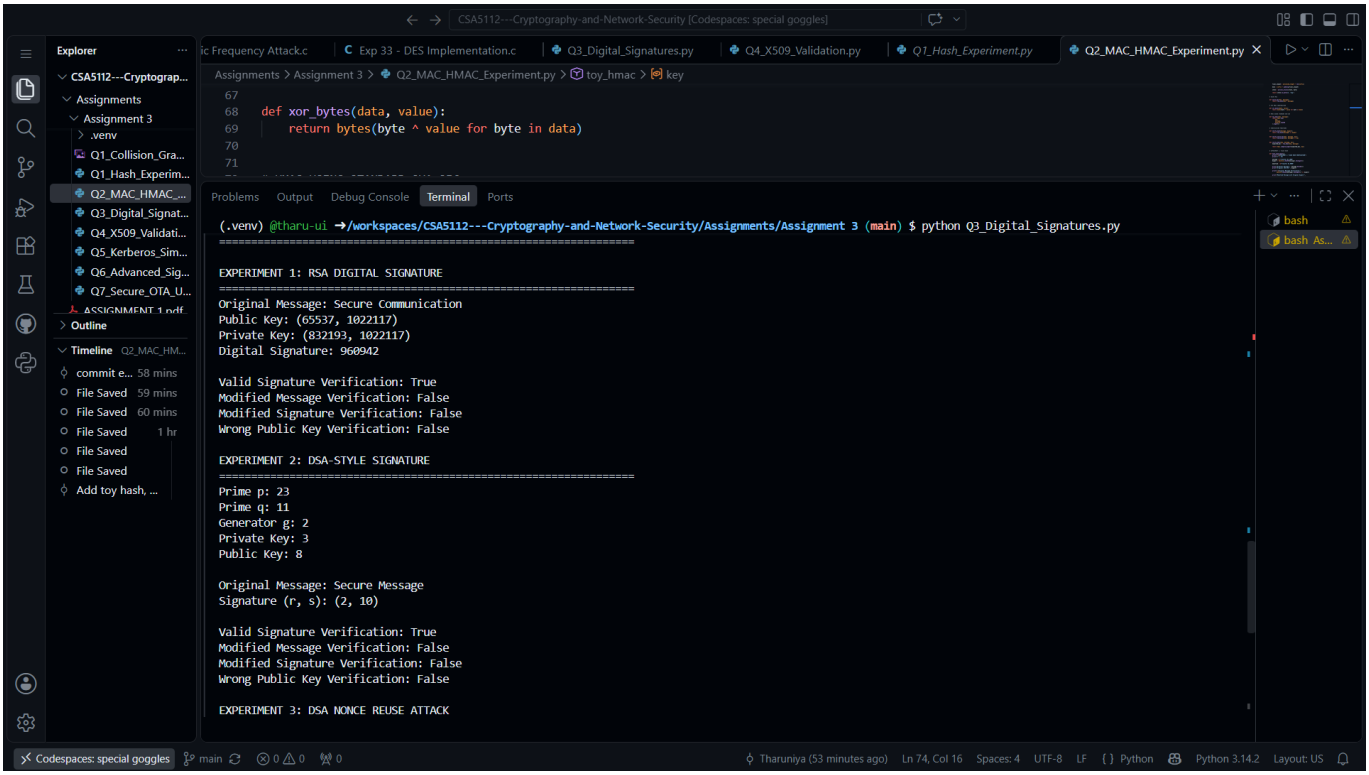
3.12 Conclusion

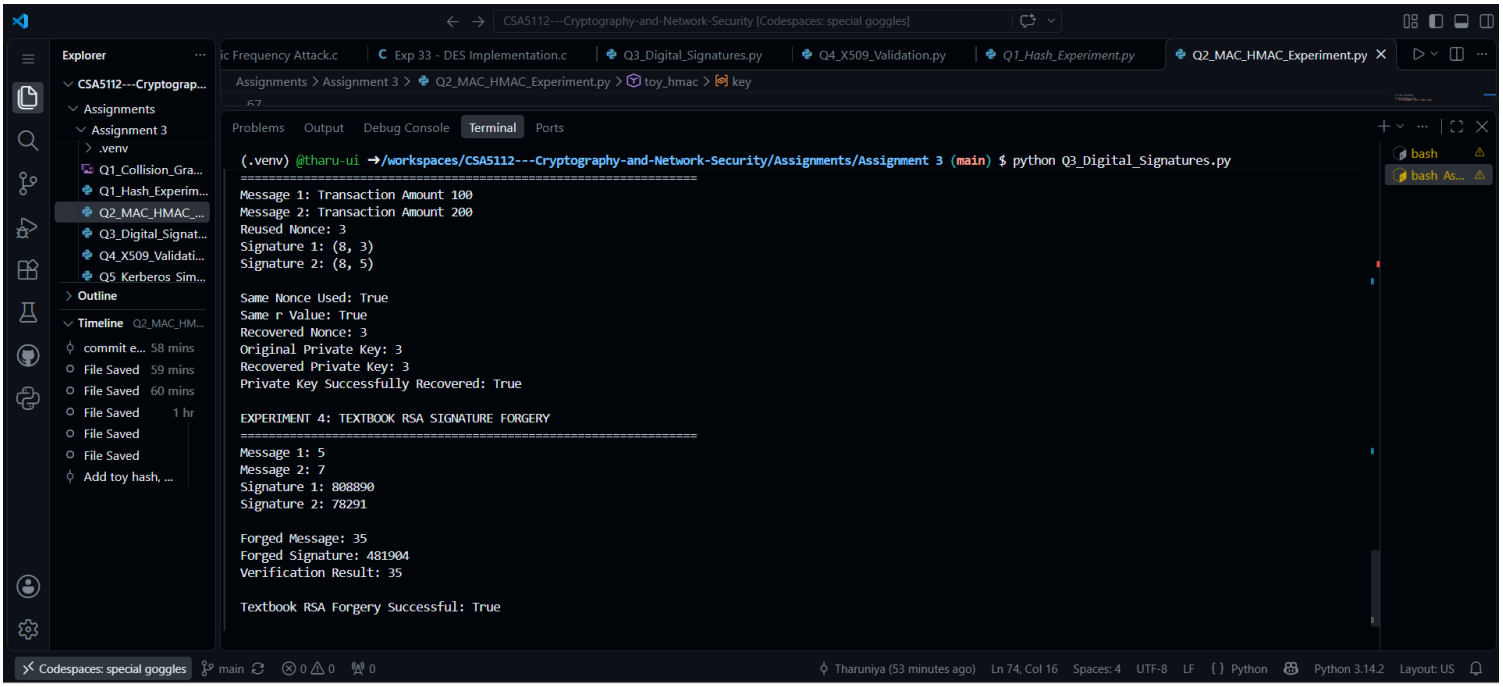
The experiment demonstrated RSA and DSA-style signature generation and verification.

Modified messages, modified signatures and incorrect public keys were rejected.

The nonce reuse experiment recovered the original DSA private key, while the textbook RSA experiment demonstrated a multiplicative forgery.

These results show that cryptographic security depends not only on mathematical algorithms but also on correct implementation, secure key management, strong randomness and appropriate signature padding.




CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
CSA5112---Cryptograp...
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Validati...
Q5_Kerberos_Sim...
Outline
Timeline Q2_MAC_HM...
commit e... 58 mins
File Saved 59 mins
File Saved 60 mins
File Saved 1 hr
File Saved
File Saved
Add toy hash, ...
ic Frequency Attack.c
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems
Output
Debug Console
Terminal
Ports
bash
bash As...
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q3_Digital_Signatures.py
==
Message 1: Transaction Amount 100
Message 2: Transaction Amount 200
Reused Nonce: 3
Signature 1: (8, 3)
Signature 2: (8, 5)
Same Nonce Used: True
Same r Value: True
Recovered Nonce: 3
Original Private Key: 3
Recovered Private Key: 3
Private Key Successfully Recovered: True
EXPERIMENT 4: TEXTBOOK RSA SIGNATURE FORGERY
==
Message 1: 5
Message 2: 7
Signature 1: 808890
Signature 2: 78291
Forged Message: 35
Forged Signature: 481904
Verification Result: 35
Textbook RSA Forgery Successful: True
Codespaces: special goggles
main
Tharuniya (53 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US

Q4 – TRUST THROUGH CERTIFICATES: X.509 CHAIN VALIDATION (7 MARKS)

Here the responsibilities are deliberately reversed so that the same student does not always perform the theoretical component.

Shared Group Work

Together:

- **Study the structure of an X.509 v3 certificate (subject, issuer, serial number, validity, public key, extensions such as key usage and basic constraints, signature)**
- **Build a small three-level hierarchy (root CA → intermediate CA → end-entity) using a library or a toolkit such as OpenSSL**
- **Implement a chain-validation procedure that follows the steps a client must perform**
- **Verify the procedure on valid and invalid chains**

Student B – Trust Model & Validation Logic
Student B must individually:

- **Write the complete validation algorithm in pseudocode (signature check on each link, issuer–subject matching, validity period, basic-constraints CA flag, key usage, name matching, revocation check, anchoring at a trusted root)**
- **Explain what each step protects against**
- **Explain why trust in the root cannot itself be verified cryptographically ● Explain how X.509 binds an identity to a public key and why that binding is what makes a signature meaningful**

Student A – Failure-Case Experiments

Student A must individually:

- **Design at least six failure cases: expired certificate, not-yet-valid certificate, wrong issuer, tampered certificate field, hostname mismatch, end-entity certificate used as a CA, revoked certificate**
- **Record which step of validation rejects each case**
- **Compare CRL and OCSP (freshness, size, privacy, availability) and show a case in which revocation is detected only if the check is actually**

performed ● Discuss what a "fail-open" revocation policy means for security

Joint Engineering Decision

Together determine:

If a CA is compromised or a certificate is mis-issued, what limits the damage, how long can the damage persist, and what would you change in a deployment to shorten that window?

4.1 Introduction

X.509 is a standard that defines the format of digital certificates used in Public Key Infrastructure (PKI).

An X.509 certificate binds an identity to a public key. It is digitally signed by a Certificate Authority (CA) or another issuing authority.

Certificates are widely used in HTTPS, TLS, secure email, digital signatures and authentication systems.

The purpose of certificate chain validation is to establish whether a certificate can be trusted through a chain of issuing certificates leading to a trusted root CA.

4.2 Structure of an X.509 Certificate

An X.509 certificate contains the following important fields:

Field	**Description**
Subject	Identity of the certificate owner
Issuer	Authority that issued the certificate
Serial Number	Unique certificate identifier
Validity Period	Start and expiry dates
Public Key	Public key belonging to the subject

Signature Algorithm	Algorithm used to sign the certificate
Basic Constraints	Indicates whether the certificate can act as a CA
Key Usage	Specifies permitted cryptographic operations
Subject Alternative Name	Contains domain names or other identities
Digital Signature	Issuer's signature over the certificate information

4.3 X.509 Certificate Hierarchy

A three-level certificate hierarchy was implemented.

The hierarchy contains:

1. Root Certificate Authority
2. Intermediate Certificate Authority
3. End-Entity Certificate

Certificate Chain Diagram

Root CA
↓ Signs
Intermediate CA
↓ Signs
End-Entity Certificate
↓
www.example.com

Root CA: The root certificate is the trust anchor. It is installed or configured as trusted by the client.

Intermediate CA: The intermediate certificate is issued by the root CA and is responsible for issuing end-entity certificates.

End-Entity Certificate: The end-entity certificate belongs to a website, server or user. It contains the public key associated with that identity.

4.4 Certificate Chain Validation Procedure

A certificate chain must be checked before a client accepts the identity presented by a server.

Validation Algorithm

1. Receive the end-entity certificate and the required intermediate certificates.
2. Identify the issuer of the end-entity certificate.
3. Check whether the issuer matches the subject of the intermediate certificate.
4. Verify the end-entity certificate's digital signature using the intermediate CA's public key.
5. Check the validity period of the certificate.
6. Check the intermediate certificate's issuer and signature.
7. Verify that the intermediate certificate is permitted to act as a CA.
8. Check key usage and other relevant certificate extensions.
9. Verify that the hostname matches the certificate identity.
10. Check certificate revocation status using CRL or OCSP where required.
11. Confirm that the chain terminates at a trusted root CA.
12. Accept the certificate only if all required validation checks succeed.

Pseudocode

```
FUNCTION ValidateCertificateChain(leaf, intermediate, root):

  IF leaf.issuer != intermediate.subject:
    REJECT

  IF intermediate.issuer != root.subject:
    REJECT

  IF current_time is outside leaf validity:
    REJECT
```

IF current_time is outside intermediate validity:

REJECT

IF leaf signature is invalid:

REJECT

IF intermediate signature is invalid:

REJECT

IF intermediate is not permitted to act as a CA:

REJECT

IF key usage is invalid:

REJECT

IF hostname does not match:

REJECT

IF certificate is revoked:

REJECT

IF root is not trusted:

REJECT

ACCEPT

4.5 Purpose of Each Validation Step

Validation Step	**Security Purpose**
Issuer–subject matching	Prevents an unrelated certificate from being inserted into the chain.

Signature verification	Detects modification of certificate information.
Validity period	Prevents acceptance of expired or not-yet-valid certificates.
Basic constraints	Prevents an end-entity certificate from being used as a CA.
Key usage	Ensures that the public key is used only for permitted purposes.
Hostname matching	Ensures that the certificate belongs to the requested website.
Revocation check	Detects certificates that have been invalidated before expiry.
Trusted root verification	Connects the certificate chain to an authority trusted by the client.

4.6 Experimental Implementation

A Python program named Q4_X509_Validation.py was implemented using the cryptography library.

The program generated a root CA, an intermediate CA and an end-entity certificate.

The certificate chain was tested against multiple valid and invalid conditions.

4.7 Experimental Results – Valid Certificate Chain

Parameter	**Result**
Root CA	CN=SIMATS Root CA
Intermediate CA	CN=SIMATS Intermediate CA
End Entity	CN=www.example.com
Certificate Chain	Root CA → Intermediate CA → End Entity
Validation Result	ACCEPTED

The valid certificate chain was successfully verified after correcting the certificate public-key handling in the implementation.

4.8 Experimental Results – Invalid Certificate Tests

Test Case	Result	Reason
Expired Certificate	REJECTED	Certificate has expired
Not-Yet-Valid Certificate	REJECTED	Certificate is not yet valid
Wrong Issuer	REJECTED	Intermediate issuer mismatch
Hostname Mismatch	REJECTED	Hostname mismatch
End Entity Used as CA	REJECTED	Intermediate certificate is not a CA
Revoked Certificate	REJECTED	Certificate has been revoked
Tampered Certificate	REJECTED	Certificate signature verification failed

Observation

All the tested invalid certificates were rejected for the corresponding validation failures.

The experiment demonstrated that certificate validation must involve more than checking whether a certificate contains a public key. The issuer, signature, validity, identity, constraints and revocation status must also be considered.

4.9 Certificate Revocation: CRL and OCSP

A certificate may need to be invalidated before its expiry date if the associated private key is compromised or the certificate has been mis-issued.

Two common revocation mechanisms are CRL and OCSP.

Certificate Revocation List (CRL)

A CRL is a digitally signed list of revoked certificate serial numbers published by a Certificate Authority.

The client downloads the list and checks whether the certificate's serial number appears in it.

Online Certificate Status Protocol (OCSP)

OCSP allows a client to query an OCSP responder about the status of an individual certificate.

The responder returns a status such as good, revoked or unknown.

Comparison of CRL and OCSP

Feature	CRL	OCSP
Method	Downloads a list of revoked certificates	Queries the status of a specific certificate
Freshness	Depends on CRL update frequency	Can provide more recent status
Size	Can become large	Usually smaller response
Privacy	Downloading a list does not directly reveal one queried certificate	The responder may learn which certificate is being checked
Availability	Requires access to an updated CRL	Requires an available OCSP responder unless a suitable response is cached
Main Limitation	Large lists may be inefficient	Responder availability and privacy concerns

4.10 Experimental Results – CRL Revocation Check

Test Case	Result
Revoked Certificate Without CRL Checking	ACCEPTED
Revoked Certificate With CRL Checking	REJECTED
CRL Contains Revoked Serial Number	True

Observation

When revocation checking was disabled, the certificate passed the simplified validation process even though it had been revoked.

When CRL checking was enabled, the revoked certificate was rejected.

This demonstrates why revocation checking is an important part of certificate validation.

4.11 Fail-Open and Fail-Closed Policies

Fail-Open Policy: If the revocation service is unavailable, the system accepts the certificate.

This improves availability but may allow a compromised or revoked certificate to be accepted.

Fail-Closed Policy: If revocation status cannot be verified, the system rejects the certificate or refuses the protected operation.

This improves security but may affect service availability.

The correct policy depends on the security requirements of the application. High-risk systems should avoid silently accepting certificates whose revocation status cannot be established.

4.12 Why Cannot the Root CA's Trust Be Verified Cryptographically?

A root CA is normally self-signed.

Its self-signature proves that the certificate was signed using the private key corresponding to its public key. However, it does not independently prove that the authority is trustworthy.

Trust in a root CA comes from an external trust decision, such as installation in an operating system's trusted root store or approval by an organisation.

Therefore, cryptography can verify the certificate chain, but the initial trust anchor must be established through a trusted process.

4.13 Limiting Damage Caused by a Compromised CA

The impact of a compromised or mis-issuing CA can be reduced through:

- Short certificate validity periods.
- Immediate revocation of affected certificates.
- Secure protection of CA private keys.
- Hardware Security Modules for important CA keys.
- Certificate Transparency monitoring.
- Restricted intermediate CA permissions.
- Regular audit and certificate issuance monitoring.

The implementation used for this assignment is an educational certificate-chain validator. It demonstrates the major validation checks but is not a complete production-grade PKIX implementation.

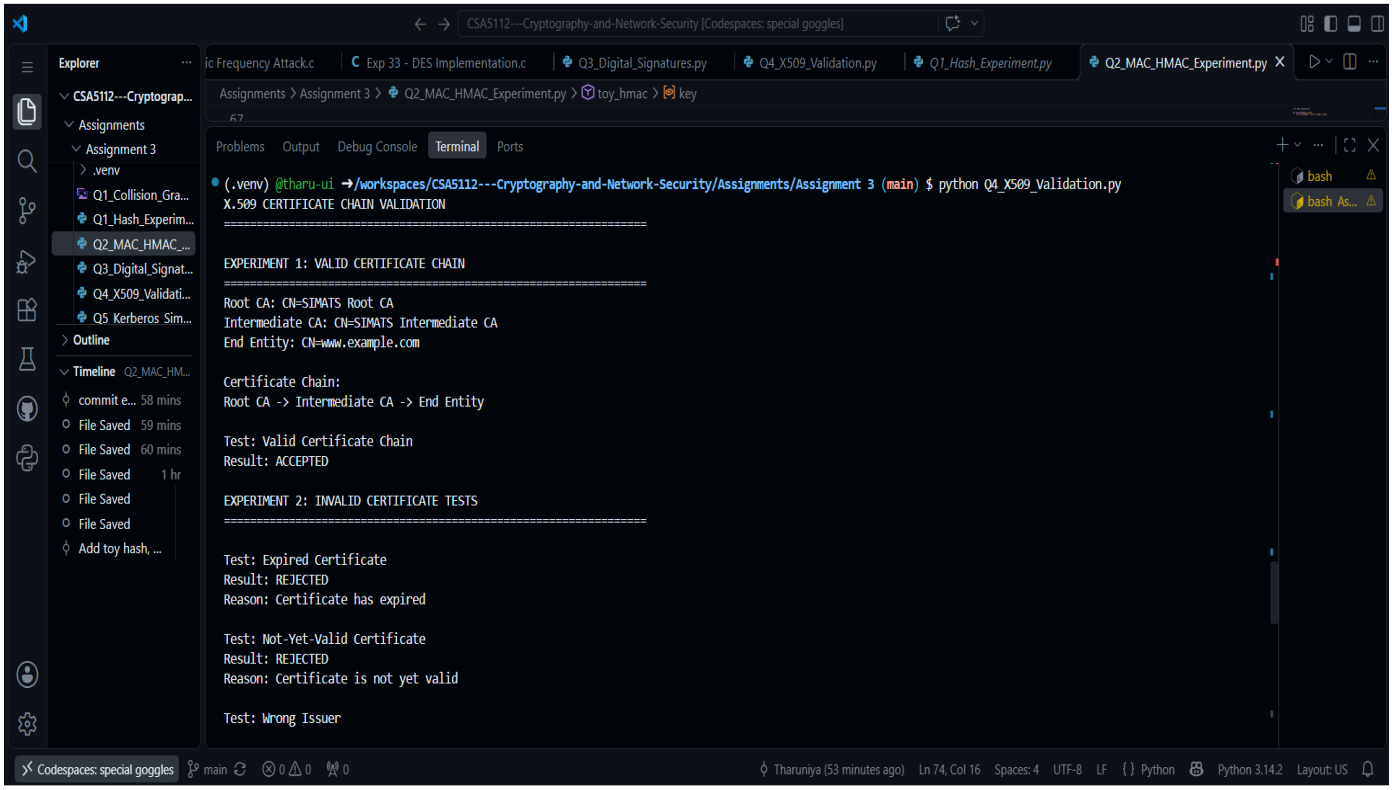
4.14 Conclusion

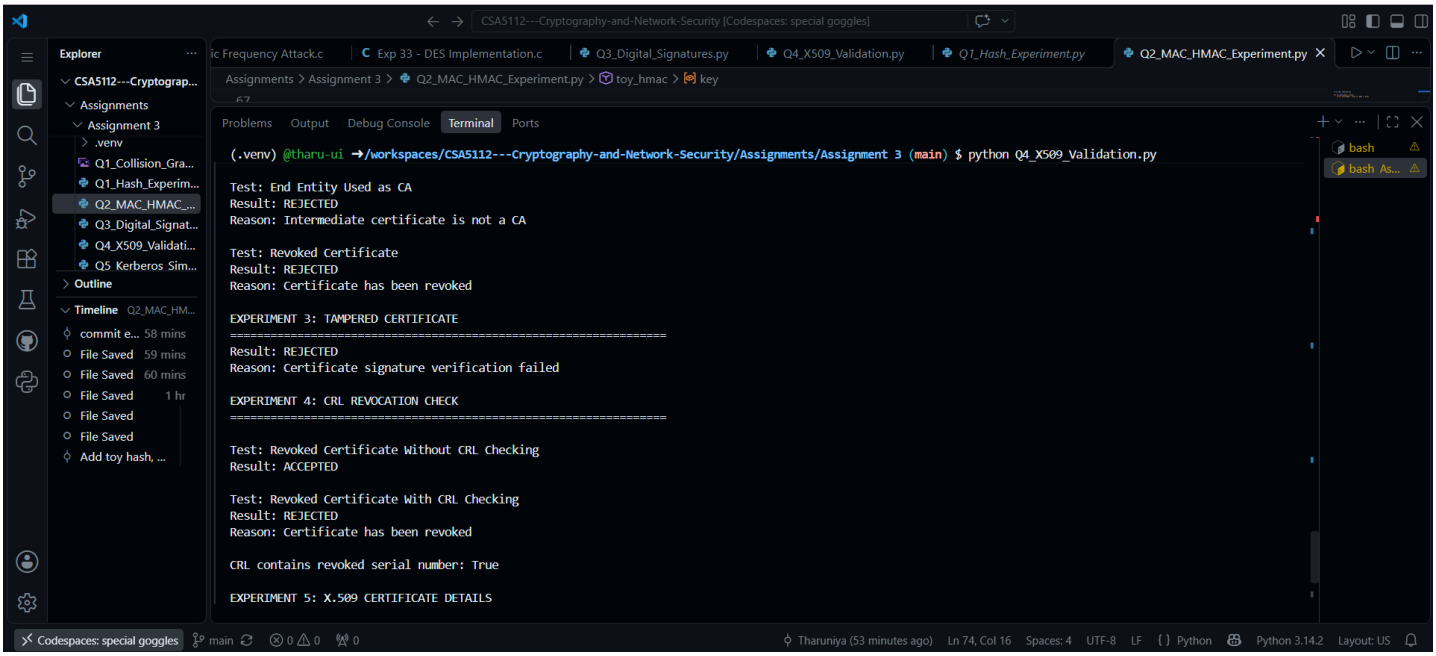
The experiment demonstrated the structure and validation of an X.509 certificate chain.

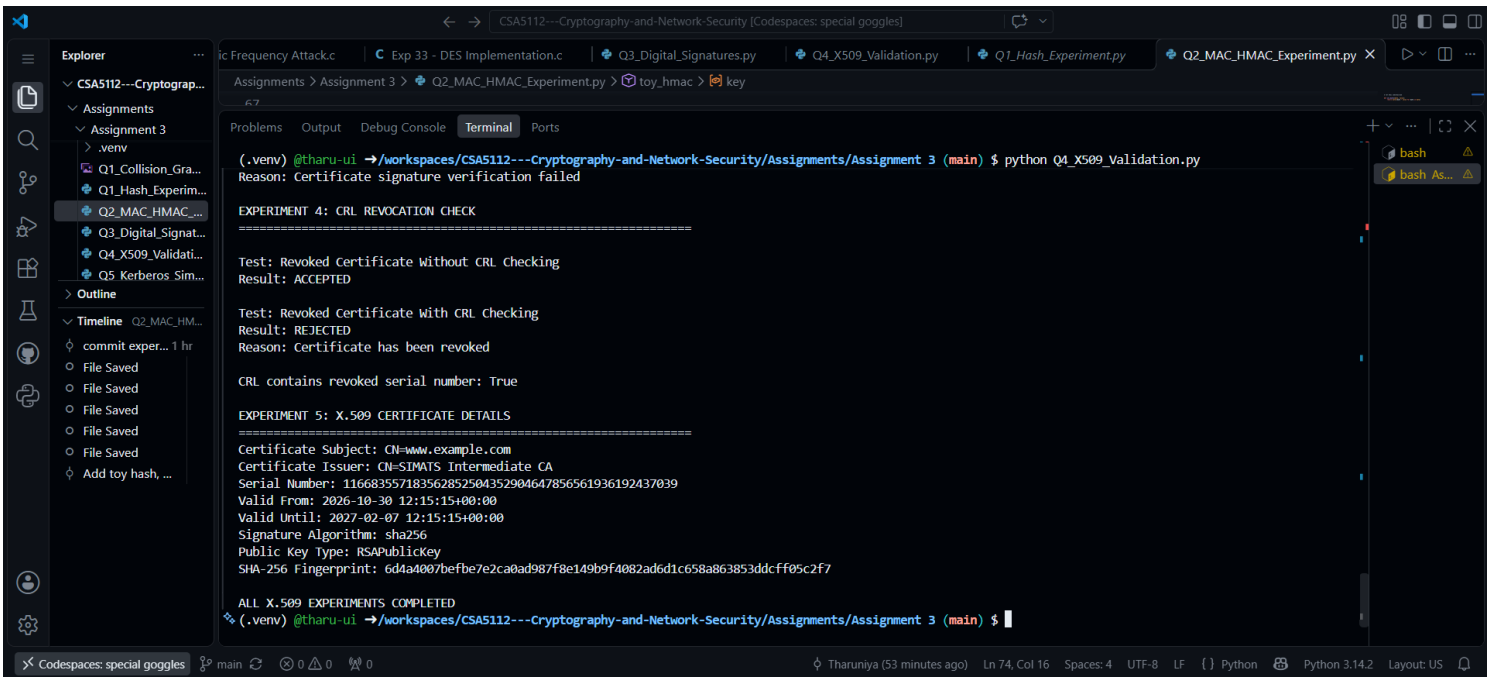
The valid certificate was accepted, while expired, not-yet-valid, incorrectly issued, tampered, mismatched and revoked certificates were rejected.

The CRL experiment demonstrated that revocation checking must actually be performed to detect revoked certificates.

X.509 provides a scalable method of binding identities to public keys, but its security depends on trusted certificate authorities, secure private-key management and correct validation policies.




CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Validati...
Q5 Kerberos Sim...
Outline
Timeline
commit e... 58 mins
File Saved 59 mins
File Saved 60 mins
File Saved 1 hr
File Saved
File Saved
Add toy hash, ...
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems Output Debug Console Terminal Ports
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q4_X509_Validation.py
Test: End Entity Used as CA
Result: REJECTED
Reason: Intermediate certificate is not a CA
Test: Revoked Certificate
Result: REJECTED
Reason: Certificate has been revoked
EXPERIMENT 3: TAMPERED CERTIFICATE
Result: REJECTED
Reason: Certificate signature verification failed
EXPERIMENT 4: CRL REVOCATION CHECK
Test: Revoked Certificate Without CRL Checking
Result: ACCEPTED
Test: Revoked Certificate With CRL Checking
Result: REJECTED
Reason: Certificate has been revoked
CRL contains revoked serial number: True
EXPERIMENT 5: X.509 CERTIFICATE DETAILS
bash
bash As...
Codespaces: special goggles
main
Tharuniya (53 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US


CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Validati...
Q5_Kerberos_Sim...
Outline
Timeline
commit exper... 1 hr
File Saved
File Saved
File Saved
File Saved
Add toy hash, ...
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems Output Debug Console Terminal Ports
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q4_X509_Validation.py
Reason: Certificate signature verification failed
EXPERIMENT 4: CRL REVOCATION CHECK
Test: Revoked Certificate Without CRL Checking
Result: ACCEPTED
Test: Revoked Certificate With CRL Checking
Result: REJECTED
Reason: Certificate has been revoked
CRL contains revoked serial number: True
EXPERIMENT 5: X.509 CERTIFICATE DETAILS
Certificate Subject: CN=www.example.com
Certificate Issuer: CN=SIMATS Intermediate CA
Serial Number: 116683557183562852504352904647856561936192437039
Valid From: 2026-10-30 12:15:15+00:00
Valid Until: 2027-02-07 12:15:15+00:00
Signature Algorithm: sha256
Public Key Type: RSAPublicKey
SHA-256 Fingerprint: 6d4a4007befbe7e2ca0ad987f8e149b9f4082ad6d1c658a863853ddcff05c2f7
ALL X.509 EXPERIMENTS COMPLETED
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $
bash
bash As...
Codespaces: special goggles
main
Tharuniya (53 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US

Q5 – KERBEROS: AUTHENTICATION WITHOUT SENDING PASSWORDS (6 MARKS)

Together:

- **Study the Kerberos message flow (Authentication Server exchange, Ticket-Granting Server exchange, Client–Server exchange)**
- **Build a simulation with a client, KDC (AS + TGS) and one service, using symmetric encryption and authenticated tickets**
- **Include timestamps, ticket lifetimes, session keys and authenticators ● Test the normal flow and the attack/failure cases below**

Student A – Protocol Design

Student A must individually:

- **Draw the message-sequence diagram and label what is encrypted with which key**
- **Explain what the Ticket-Granting Ticket is for and why the user's long-term key is used only once per session**
- **Explain the purpose of the authenticator and the timestamp**
- **Explain how mutual authentication is achieved**
- **Identify which party can read which part of each message**

Student B – Attack and Failure Evidence

Student B must individually:

- **Demonstrate a replay of a captured authenticator with and without replay protection**
- **Demonstrate an expired ticket and a clock-skew failure**
- **Demonstrate what happens if the KDC is unavailable**
- **Record accept/reject outcomes and the reason for each**
- **Explain what a stolen ticket or an offline password-guessing attack on the AS reply would allow**

Cross-Understanding Requirement

Student B must be able to explain:

Why does the client never send its password across the network, and what does the service learn about the client?

Student A must be able to explain:

What experimental evidence shows that the timestamp/authenticator mechanism actually prevents replay?This deliberately tests understanding across the assigned responsibilities.

5.1 Introduction

Kerberos is a network authentication protocol that uses symmetric-key cryptography and a trusted third party called the Key Distribution Center (KDC).

It allows users and services to authenticate each other without transmitting the user's password across the network.

Kerberos uses tickets, session keys, authenticators and timestamps to provide secure authentication and help prevent replay attacks.

5.2 Components of Kerberos

1. Client

The client is the user or system requesting access to a network service.

2. Authentication Server (AS)

The AS verifies the client's initial authentication request and provides a Ticket-Granting Ticket.

3. Ticket-Granting Server (TGS)

The TGS issues service tickets after receiving a valid Ticket-Granting Ticket and authenticator.

4. Application Server

The application server provides the requested service and verifies the client's service ticket.

5. Key Distribution Center (KDC)

The KDC contains the Authentication Server and Ticket-Granting Server.

5.3 Kerberos Authentication Process

Kerberos authentication takes place through three major exchanges:

1. Authentication Server Exchange.
2. Ticket-Granting Server Exchange.
3. Client–Server Exchange.

Kerberos Message-Sequence Diagram

Stage 1 – Authentication Server Exchange

Client → AS: Authentication request.

AS → Client: Ticket-Granting Ticket and client–TGS session key.

The client decrypts the protected response using information derived from its long-term secret.

Stage 2 – Ticket-Granting Server Exchange

Client → TGS: Ticket-Granting Ticket, authenticator and requested service.

TGS → Client: Service ticket and client–server session key.

Stage 3 – Client–Server Exchange

Client → Application Server: Service ticket and authenticator.

Application Server → Client: Server authentication response.

Client verifies the response and gains access to the requested service.

5.4 Purpose of the Ticket-Granting Ticket

The Ticket-Granting Ticket (TGT) allows a client to request additional service tickets without repeatedly entering its password.

The TGT is encrypted using a key known to the TGS.

The client cannot modify the protected ticket information.

The TGT contains information such as the client's identity, session-key information and ticket lifetime.

The user's long-term secret is used during the initial authentication process rather than being transmitted repeatedly for every service request.

5.5 Purpose of the Authenticator and Timestamp

An authenticator is a short-lived data structure generated by the client using the client–server session key.

It typically contains the client's identity and a timestamp.

The timestamp helps the server determine whether the request is recent.

The authenticator helps prevent replay attacks because a captured request should not be accepted repeatedly.

A server can also maintain a replay cache containing previously accepted authenticators.

5.6 Mutual Authentication

Mutual authentication means that both the client and server verify each other's identity.

The client first sends the service ticket and authenticator to the application server.

The server validates the ticket and authenticator.

The server then returns an authenticated response, commonly based on the timestamp in the authenticator.

The client verifies this response using the session key.

If verification succeeds, the client gains assurance that it is communicating with the intended service.

5.7 Who Can Read Each Message?

Message or Information	**Who Can Read It?**
Initial authentication request	AS can process the request

Protected AS response	Client can decrypt the client-specific portion; TGS can read the TGT
Ticket-Granting Ticket	TGS can decrypt the ticket
Service ticket	Application server can decrypt the ticket
Authenticator	Intended receiving server can decrypt and verify it using the session key
Service response	Client can verify it using the session key

The password itself is not transmitted directly across the network.

5.8 Experimental Implementation

A Python program named Q5_Kerberos_Simulation.py was implemented to demonstrate the Kerberos authentication flow.

The simulation included:

- Client.
- Authentication Server.
- Ticket-Granting Server.
- Application Server.
- Session keys.
- Tickets.
- Authenticators.
- Timestamps.
- Replay protection.

The program tested normal authentication and several failure scenarios.

5.9 Experimental Results – Normal Authentication

Step	Operation	Result
1	Client requests authentication from AS	Accepted
2	Client receives TGT	Received
3	Client requests a service ticket from TGS	Generated
4	Client receives service ticket	Received
5	Client sends ticket and authenticator	Authenticated
6	Client verifies server response	True
	Final Result	**ACCESS GRANTED**

Observation

The normal authentication flow completed successfully.

The client received the required tickets and session keys, and the application server authenticated the client.

Mutual authentication was also successful.

5.10 Replay Attack Experiment

A replay attack occurs when an attacker captures a valid authentication message and attempts to reuse it later.

The experiment compared authentication without replay protection and authentication with replay protection.

Test Case	First Request	Replayed Request
Without Replay Protection	ACCEPTED	ACCEPTED
With Replay Protection	ACCEPTED	REJECTED

Observation

Without replay protection, the captured authenticator was accepted again.

With replay protection, the first request was accepted and the repeated request was rejected.

This demonstrates the importance of timestamps and replay-cache mechanisms in Kerberos.

5.11 Experimental Results – Failure Cases

Test Case	Result	Reason
Expired TGT	REJECTED	Ticket-Granting Ticket has expired
Clock Skew	REJECTED	Clock skew exceeded
KDC Unavailability	REJECTED	KDC is unavailable

Observation

The expired ticket was rejected because its validity period had ended.

The clock-skew test was rejected because the timestamp difference exceeded the allowed limit.

When the KDC was unavailable, the simulation could not complete the required authentication or ticket-issuing operation.

5.12 Security Risks in Kerberos

Stolen Ticket

If an attacker steals a valid ticket, the attacker may attempt to impersonate the client until the ticket expires, especially if the associated session key or authenticator protections are also compromised.

Short ticket lifetimes, secure endpoint storage and replay protection reduce this risk.

Offline Password-Guessing Attack

If an attacker captures certain password-derived encrypted responses, the attacker may attempt offline password guesses.

A weak user password makes such attacks more practical.

Strong passwords, appropriate encryption and secure pre-authentication mechanisms help reduce this risk.

KDC as a Single Point of Failure

The KDC is central to Kerberos authentication. If it becomes unavailable, new authentication and ticket-issuing operations may fail.

High availability, replication and secure KDC administration are therefore important.

5.13 Why Does the Client Never Send Its Password Across the Network?

The client uses its long-term secret to decrypt or verify protected authentication information received from the AS.

The password is not sent directly as a network message.

Instead, Kerberos uses encrypted tickets and session keys to authenticate the client and protect subsequent communication.

The application server learns the authenticated identity represented in the ticket, along with the authorization information available to it. It does not need to know the user's password.

5.14 Conclusion

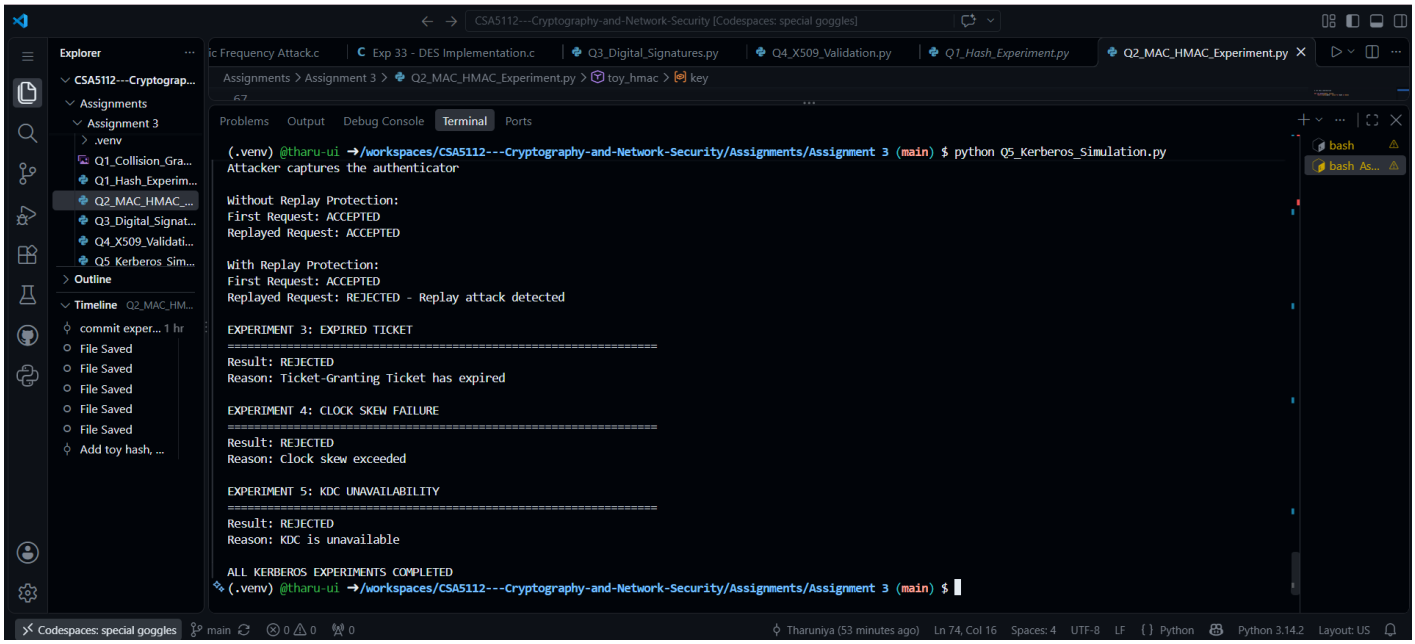
The experiment demonstrated the normal Kerberos authentication flow and the use of tickets, authenticators, timestamps and session keys.

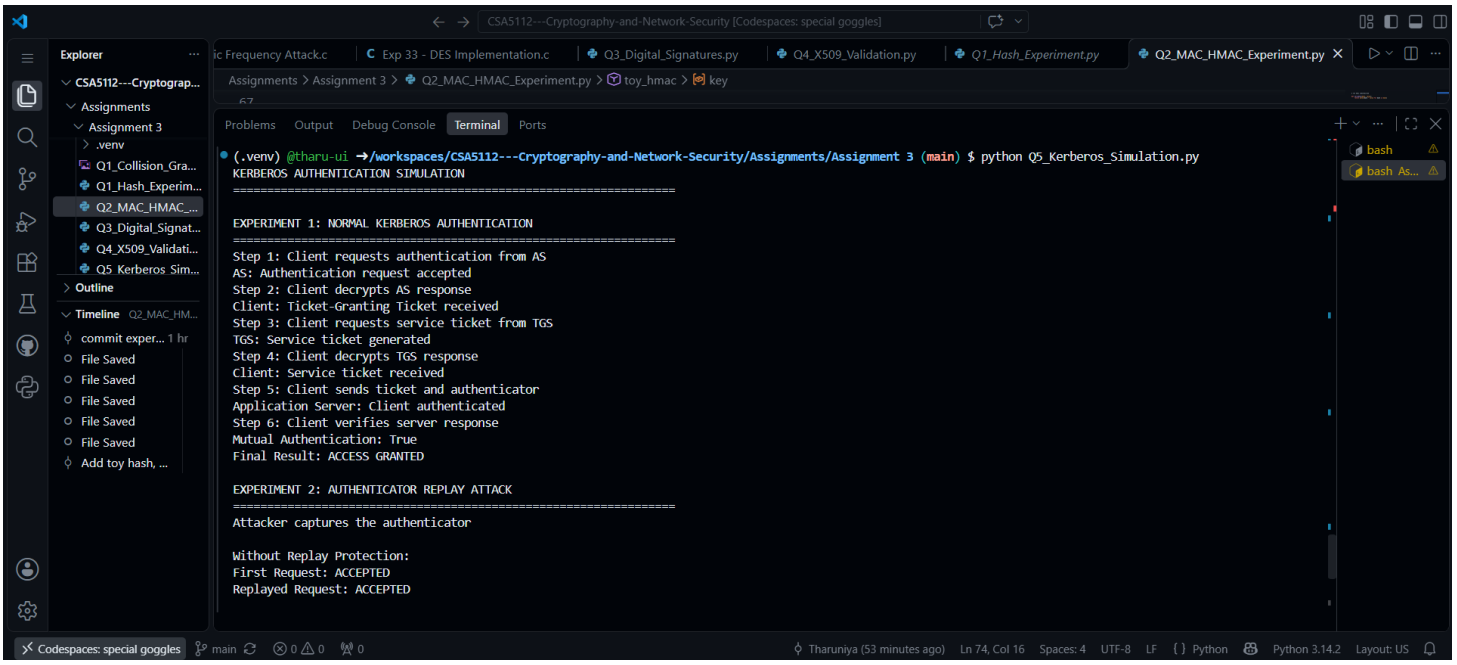
The replay experiment showed that replay protection prevented a captured authenticator from being accepted a second time.

Expired tickets, excessive clock skew and KDC unavailability were also tested successfully.

Kerberos provides efficient centralized authentication for network environments, but its security depends on protecting the KDC, managing tickets and maintaining secure session keys.

The program is an educational simulation and does not implement the complete standardized Kerberos protocol.


(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q5_Kerberos_Simulation.py
Attacker captures the authenticator
Without Replay Protection:
First Request: ACCEPTED
Replayed Request: ACCEPTED
With Replay Protection:
First Request: ACCEPTED
Replayed Request: REJECTED - Replay attack detected
EXPERIMENT 3: EXPIRED TICKET
Result: REJECTED
Reason: Ticket-Granting Ticket has expired
EXPERIMENT 4: CLOCK SKEW FAILURE
Result: REJECTED
Reason: Clock skew exceeded
EXPERIMENT 5: KDC UNAVAILABILITY
Result: REJECTED
Reason: KDC is unavailable
ALL KERBEROS EXPERIMENTS COMPLETED
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $


(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q5_Kerberos_Simulation.py
KERBEROS AUTHENTICATION SIMULATION
EXPERIMENT 1: NORMAL KERBEROS AUTHENTICATION
Step 1: Client requests authentication from AS
AS: Authentication request accepted
Step 2: Client decrypts AS response
Client: Ticket-Granting Ticket received
Step 3: Client requests service ticket from TGS
TGS: Service ticket generated
Step 4: Client decrypts TGS response
Client: Service ticket received
Step 5: Client sends ticket and authenticator
Application Server: Client authenticated
Step 6: Client verifies server response
Mutual Authentication: True
Final Result: ACCESS GRANTED
EXPERIMENT 2: AUTHENTICATOR REPLAY ATTACK
Attacker captures the authenticator
Without Replay Protection:
First Request: ACCEPTED
Replayed Request: ACCEPTED

Q6 – ADVANCED SECURITY MINI-INVESTIGATIONS: RSA, DSA/DSS, ECDSA, ELGAMAL AND SCHNORR (7 MARKS)

Student A – Hash Weakness and Signature Forgery

Student A takes primary responsibility for the collision-to-forgery investigation.

The student must:

- **Explain the difference between a collision attack and a second pre-image attack**
- **Using your truncated-hash collision finder from Q1, build two different messages with the same short digest and show that a signature on one is valid for the other (use a signature scheme over the truncated digest)**
- **Explain why this attack requires the attacker to prepare both messages in advance**
- **Explain how a collision in MD5/SHA-1 enabled real-world forged certificates and what this means for X.509**
- **Explain the role of digest length and algorithm choice in deciding what a CA may use**
- **Recommend a hash policy for signatures and justify it**

Student B Verification

Student B must provide a short explanation of:

Why does weakening the hash function weaken the signature, even when the signature algorithm itself (RSA/DSA) is unchanged?

Student B – Kerberos versus X.509 and Non-Repudiation

Student B takes primary responsibility for the framework comparison investigation.

The student must:

- **Compare Kerberos and X.509/PKI in a table: key type, trusted third party, online/offline dependence, scalability, revocation, typical environment ● Explain which framework gives non-repudiation and which does not, and why (shared secret vs private key held by one party)**
- **Analyse the single point of failure of Kerberos and the trust-anchor risk in X.509**
- **Compare the number of key relationships required in a network of n users for pairwise shared keys, Kerberos with a KDC, and PKI**
- **Recommend which framework you would use for (i) a campus single sign-on system and (ii) signing legally binding electronic contracts**

Student A Verification

Student A must provide a short explanation of:

Why can a Kerberos ticket prove who a user is to a service, yet not prove to a third party later that the user performed an action?

Thus, Q6 provides different primary tasks but mandatory cross-understanding.

Evidence Expected: worked toy example, verification trace, attack/failure cases and a comparison table covering RSA, DSA/DSS, ElGamal and Schnorr. • Comparison requirement: Compare RSA, DSA/DSS, ElGamal and Schnorr signatures with respect to key material, randomness/nonce requirements, verification, forgery resistance and non-repudiation assumptions. • Schnorr protocol: Describe the commitment–challenge–response sequence, derive the verification relation for toy parameters, and explain why the challenge must be unpredictable and why nonce reuse is dangerous.

• ElGamal signature scheme: Derive key generation, signing and verification for small toy parameters; test a valid signature and at least two altered-message/signature cases; explain how poor handling of signing randomness can weaken security.

6.1 Introduction

Digital signature algorithms provide message integrity and authentication through asymmetric cryptography.

Different algorithms use different mathematical problems and signing procedures.

This investigation compares RSA, DSA/DSS, ECDSA, ElGamal and Schnorr signatures. It also demonstrates collision-based signature forgery, nonce reuse and the differences between Kerberos and X.509.

6.2 Comparison of Digital Signature Algorithms

Feature	**RSA**	**DSA/DSS**	**ECDSA**	**ElGamal**	**Schnorr**
Mathematical Basis	Integer factorisation	Discrete logarithm	Elliptic-curve discrete logarithm	Discrete logarithm	Discrete logarithm
Key Material	Public and private RSA exponents	Public parameters and private exponent	Elliptic-curve public and private keys	Public and private exponents	Public and private exponents
Randomness	Standard RSA-PSS uses secure random salt	Requires secure nonce	Requires secure nonce	Requires secure nonce	Requires secure nonce
Signature Format	Single integer	Pair (r, s)	Pair (r, s)	Pair (r, s)	Depends on construction
Verification	Public-key exponentiation and encoded digest checking	Recalculation of verification values	Elliptic-curve point operations	Modular arithmetic	Challenge-response verification

Main Risk	Insecure padding or weak keys	Nonce reuse	Nonce reuse	Nonce reuse	Nonce reuse or predictable challenge
Non-Repudiation	Can support it	Can support it	Can support it	Can support it	Can support it

All these algorithms require secure key management, strong cryptographic parameters and trusted public-key identity verification.

6.3 RSA Signature Experiment

The RSA signature experiment used:

Message: Secure Digital Transaction

Public Key: (17, 1022117)

Private Key: (180017, 1022117)

Signature: 168985

Test Case	**Result**
Valid Signature	True
Modified Message	False

The original message was successfully verified. The modified message failed verification.

This demonstrates that a digital signature binds the signed digest to the message.

6.4 DSA/DSS Signature Experiment

DSA uses modular arithmetic and the discrete logarithm problem.

The experiment used the following parameters:

Parameter	**Value**
Prime p	23

Prime q	11
Generator g	2
Private Key	3
Public Key	8
Message	Digital Signature Scheme
Signature (r, s)	(2, 10)

Test Case	**Result**
Valid Signature	True
Modified Message	False

The experiment showed that the original signature was valid and the modified message was rejected.

6.5 ECDSA Signature Experiment

ECDSA is based on the elliptic-curve discrete logarithm problem.

It provides strong security with comparatively smaller key sizes than traditional RSA.

The experiment used the elliptic curve:

$y^2 = x^3 + 2x + 2 \bmod 17$

Generator point: (5, 1)

Private Key: 7

Public Key: (0, 6)

Message: Elliptic Curve Digital Signature

Signature: (10, 2)

Test Case	**Result**
Valid Signature	True

Modified Message	False

The experiment demonstrated successful verification of the original message and rejection of the modified message.

6.6 ElGamal Digital Signature

ElGamal signatures are based on the discrete logarithm problem.

Key Generation

Select a prime p and a generator alpha.

Choose a private key x.

Calculate the public key:

y = alpha^x mod p

Signing

Choose a random nonce k such that k is relatively prime to p − 1.

Calculate:

r = alpha^k mod p

s = k^(-1) × (H(m) − x × r) mod (p − 1)

The signature is the pair (r, s).

Verification

The signature is valid if:

alpha^H(m) mod p = y^r × r^s mod p

Experimental Results

Parameter	**Value**
Prime p	467
Generator alpha	2
Private Key	127

Public Key	132
Message	ElGamal Secure Message
Signature (r, s)	(32, 171)

Test Case	**Result**
Valid Signature	True
Modified Message	False
Modified Signature	False

Observation

The original ElGamal signature was accepted, while both the modified message and modified signature were rejected.

This demonstrates the integrity protection provided by signature verification.

6.7 ElGamal Nonce Reuse Experiment

ElGamal requires a fresh, unpredictable signing nonce.

Reusing the same nonce produces the same r value in signatures generated under the same key.

This creates mathematical relationships between the signatures that may allow the private key to be recovered.

Experimental Results

Parameter	**Result**
Message 1	Payment Rs.1000
Message 2	Payment Rs.5000
Signature 1	(32, 391)
Signature 2	(32, 96)

Nonce Reused	5
First r Value	32
Second r Value	32
Same r Value	True

Observation

Both signatures produced the same r value because the same nonce was reused.

The experiment demonstrated the relationship that can expose the private key to recovery attacks. The program identifies the repeated nonce and its security implications; it does not calculate the ElGamal private key in this experiment.

Therefore, every signature must use a unique and unpredictable nonce.

6.8 Schnorr Digital Signature

Schnorr signatures use a discrete logarithm-based construction.

The signing process consists of three major stages:

1. Commitment.
2. Challenge.
3. Response.

Key Generation

Select public parameters p, q and g.

Choose a private key x.

Calculate the public key:

y = g^x mod p

Commitment

The signer chooses a random nonce k and calculates:

R = g^k mod p

Challenge

The signer calculates a challenge using the commitment and message:

$e = H(R \| m) \bmod q$

Response

The signer calculates:

$s = k + e \times x \bmod q$

The signature consists of the commitment, challenge and response.

Verification Relation

The verifier calculates:

$R' = g^s \times y^{(-e)} \bmod p$

The signature is valid if the reconstructed commitment matches the commitment used to calculate the challenge.

The verifier also checks that the challenge corresponds to the message and reconstructed commitment.

The challenge must be unpredictable and bound to the message. Reusing the same nonce can reveal relationships between signatures and may expose the private key.

Experimental Results

Parameter	**Value**
Prime p	23
Prime q	11
Generator g	2
Private Key	3
Public Key	8
Message	Schnorr Authentication Message
Commitment	13

Challenge	6
Response	3

Signature: (13, 6, 3)

Test Case	**Result**
Valid Signature	True
Modified Message	False

The original signature was successfully verified, while the modified message was rejected.

6.9 Collision-Based Signature Forgery

A collision attack occurs when an attacker finds two different messages that produce the same hash value.

A second pre-image attack is different because the attacker is given one specific message and must find another message with the same digest.

A collision attack can be used against a digital signature system if the attacker prepares two different messages with the same digest before obtaining the signature.

Experimental Procedure

The program used a truncated hash to find two different messages with the same digest.

A signature was generated for the first message.

The same signature was then verified against the second message

Experimental Results

Parameter	**Result**
Message 1	Message_6
Message 2	Message_209
Truncated Hash	59183
Collision Found	True

Signature of Message 1	330286
Verification for Message 1	True
Verification for Message 2	True
Collision-Based Forgery Successful	True

Observation

The two messages produced the same truncated hash.

The signature generated for the first message was also accepted for the second message because both messages had the same digest under the experimental construction.

This demonstrates how a weak hash function can undermine an otherwise functioning signature algorithm.

The attacker must prepare both messages in advance so that the harmless-looking message can be signed before the corresponding malicious message is presented.

6.10 Real-World Implications of MD5 and SHA-1 Collisions

MD5 and SHA-1 have both suffered practical collision attacks.

A successful collision attack against a certificate-signing workflow can allow an attacker to create two different certificate-related documents with the same digest under a vulnerable signing process.

If a CA signs one document, the signature may also be valid for the other colliding document.

This can undermine the trust provided by X.509 certificates.

Modern certificate authorities should use secure hash algorithms such as SHA-256 or SHA-3 and approved signature schemes.

6.11 Why Does Weakening the Hash Weaken the Signature?

A digital signature normally authenticates the hash of a message rather than the entire message.

If an attacker can create two different messages with the same digest, the signature algorithm cannot distinguish between them based on that digest.

Therefore, even if RSA or DSA is mathematically secure, a weak hash function can allow an attacker to transfer a valid signature from one message to another.

The overall security of a digital signature system depends on both the signature algorithm and the hash function.

6.12 Kerberos versus X.509/PKI

Kerberos and X.509 are two different authentication frameworks.

Kerberos uses symmetric-key cryptography and a centralized Key Distribution Center.

X.509 uses public-key cryptography and certificates issued by trusted Certificate Authorities.

Comparison Table

Feature	**Kerberos**	**X.509 / PKI**
Key Type	Symmetric keys	Public and private keys
Trusted Third Party	Key Distribution Center	Certificate Authority
Online Dependence	KDC required for ticket issuance	Certificate checks can be performed offline using available chain information
Scalability	Centralized KDC	Distributed certificate hierarchy
Revocation	Ticket and key management	CRL and OCSP
Typical Environment	Enterprise and campus SSO	Web, TLS and digital signing
Non-Repudiation	Not normally provided	Can support non-repudiation

6.13 Non-Repudiation in Kerberos and X.509

Kerberos generally does not provide non-repudiation because it uses shared symmetric keys.

Both the client and the trusted authentication infrastructure may possess or be able to use relevant shared secrets. Therefore, a Kerberos ticket is not normally independent proof to a third party that a particular user personally performed an action.

X.509 supports public-key digital signatures.

A private key is used to create the signature, while the corresponding public key is used for verification.

When the private key is securely controlled and reliably associated with the signer's identity, a digital signature can provide evidence that the signer authorized the signed information.

However, non-repudiation also depends on key protection, certificate validity, timestamps and trustworthy audit procedures.

6.14 Single Point of Failure and Trust-Anchor Risks

Kerberos Risk

The KDC is central to the authentication system. If the KDC is compromised, attackers may be able to impersonate users or services. If it becomes unavailable, new authentication and ticket-issuing operations may fail.

Replication, backups, secure administration and strong access controls can reduce this risk.

X.509 Risk

A compromised or mis-issuing CA can issue certificates that falsely associate an identity with an attacker's public key.

This can undermine trust in certificates issued under that authority.

Restricted CA permissions, protected signing keys, certificate transparency and prompt revocation help limit the damage.

6.15 Key Relationship Comparison

For a network containing n users:

Pairwise Shared Keys

Every user must share a unique key with every other user.

Number of keys = $n(n - 1) / 2$

For 100 users:

$100 \times 99 / 2 = 4950$ keys

Kerberos with a KDC

Each user requires a long-term key shared with the KDC.

For 100 users, the client–KDC long-term key relationships are 100.

PKI

Each user maintains a public–private key pair.

For 100 users, the system requires 100 user key pairs, along with certificates and trust-management infrastructure.

Experimental Comparison

Network Size	Pairwise Shared Keys	Kerberos Client–KDC Keys	PKI User Key Pairs
100 users	4950	100	100

This demonstrates the scalability advantage of centralized Kerberos and public-key infrastructure compared with pairwise shared-key arrangements.

6.16 Recommendations

Campus Single Sign-On

Kerberos is suitable for campus single sign-on because it provides centralized authentication, ticket-based access and efficient reuse of authentication across multiple services.

Legally Binding Electronic Contracts

X.509/PKI-based digital signatures are more appropriate because they support public verification and can provide evidence of signing when private keys are securely controlled and supported by reliable identity verification and timestamping.

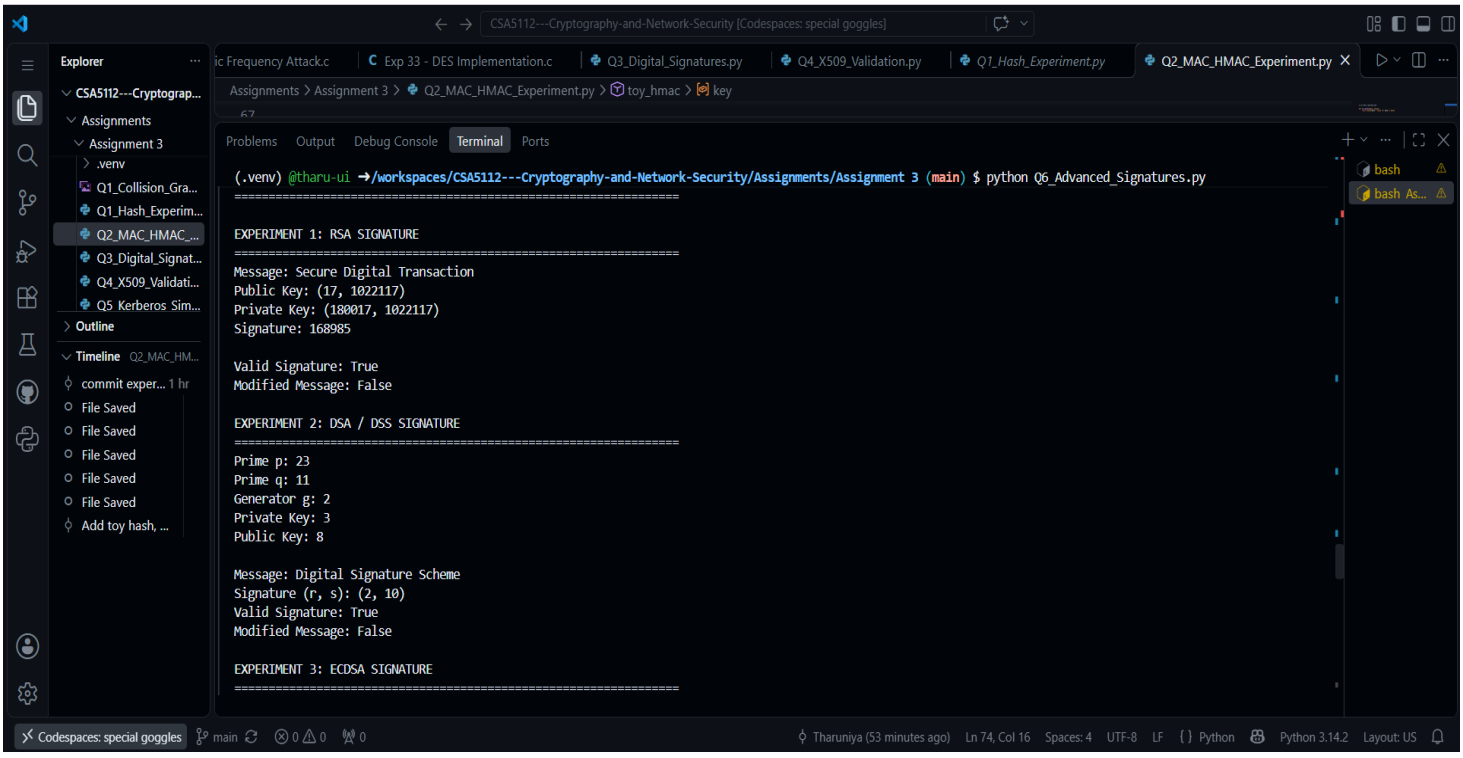
6.17 Conclusion

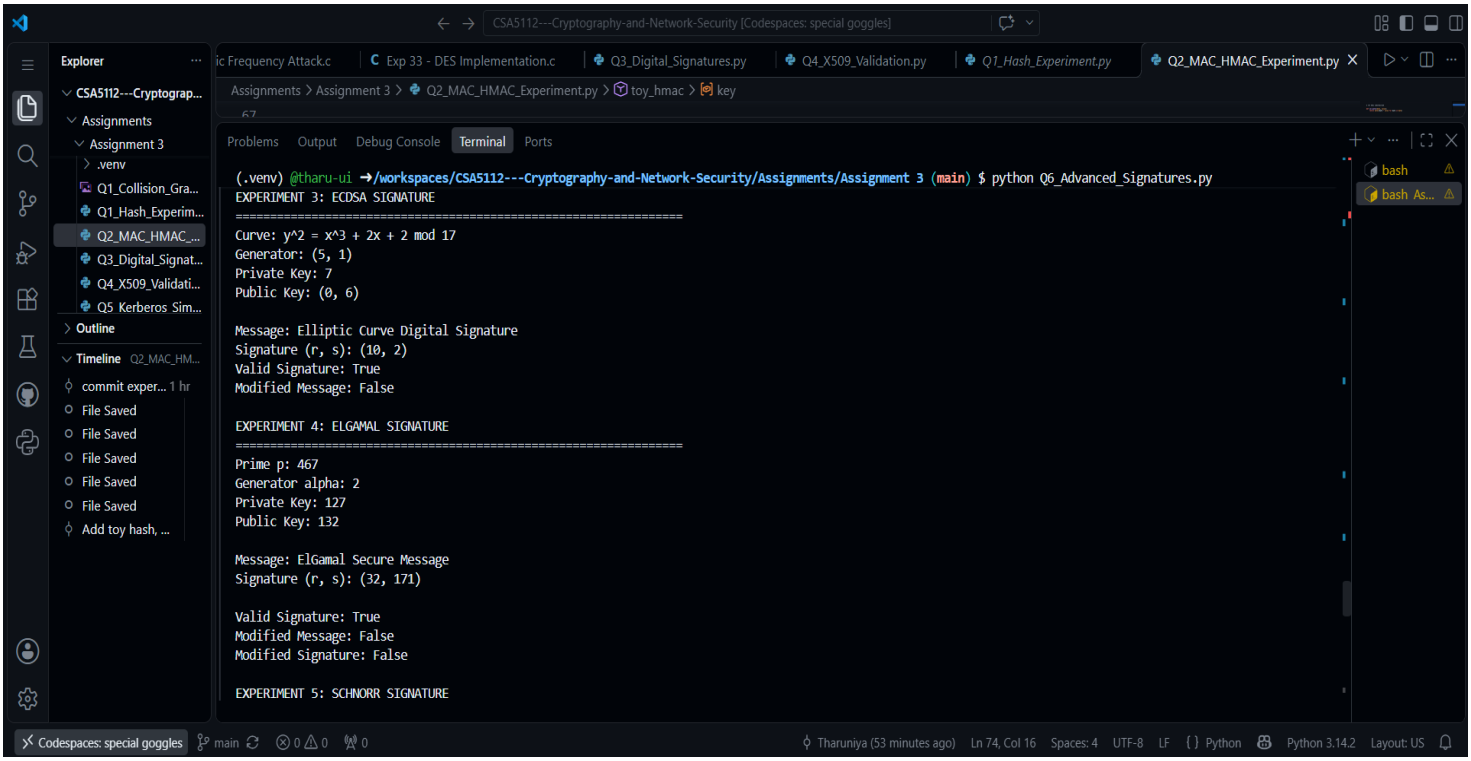
The experiments demonstrated the operation of RSA, DSA, ECDSA, ElGamal and Schnorr signatures using educational parameters.

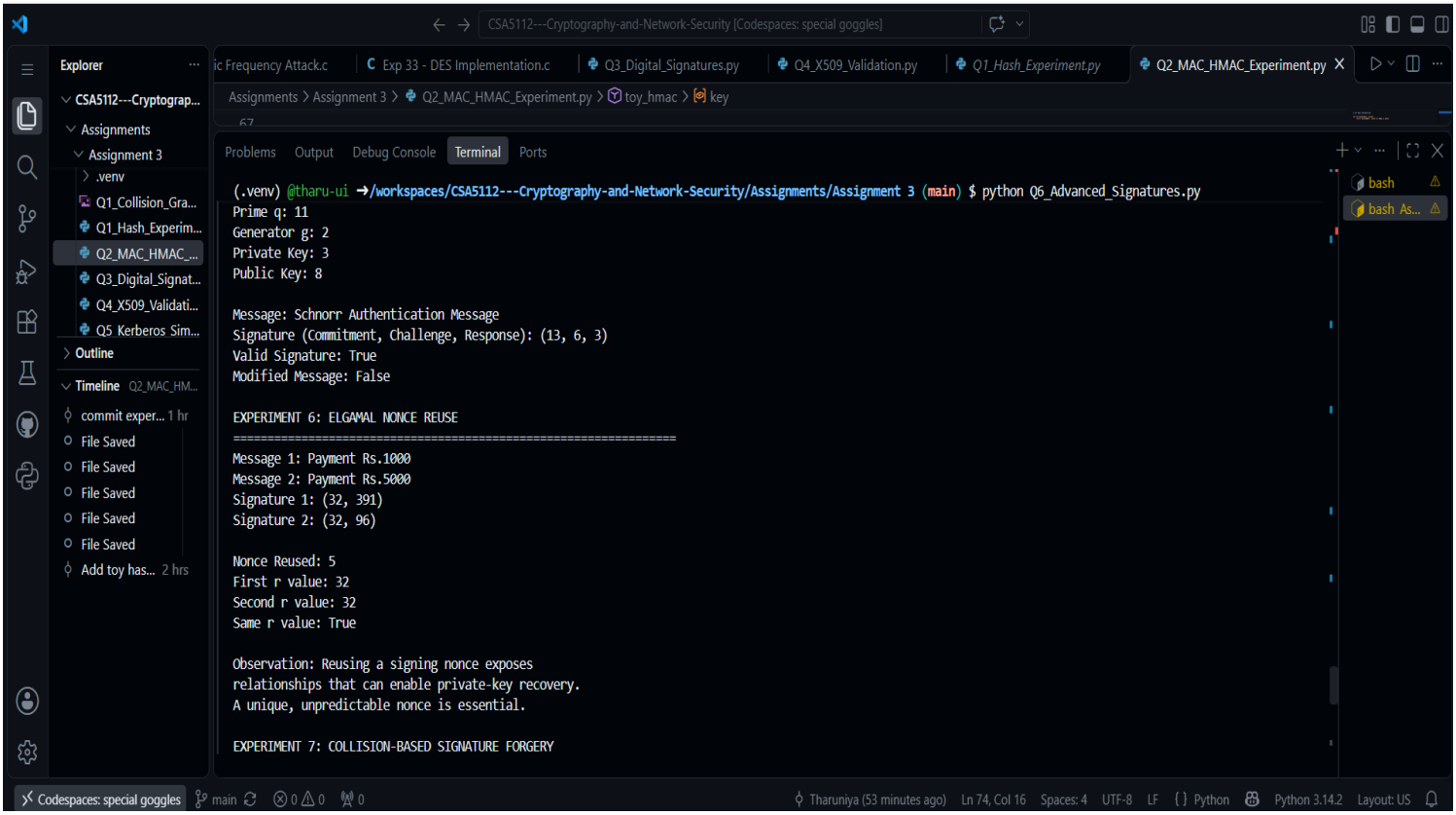
Modified messages and signatures were rejected in the corresponding tests.

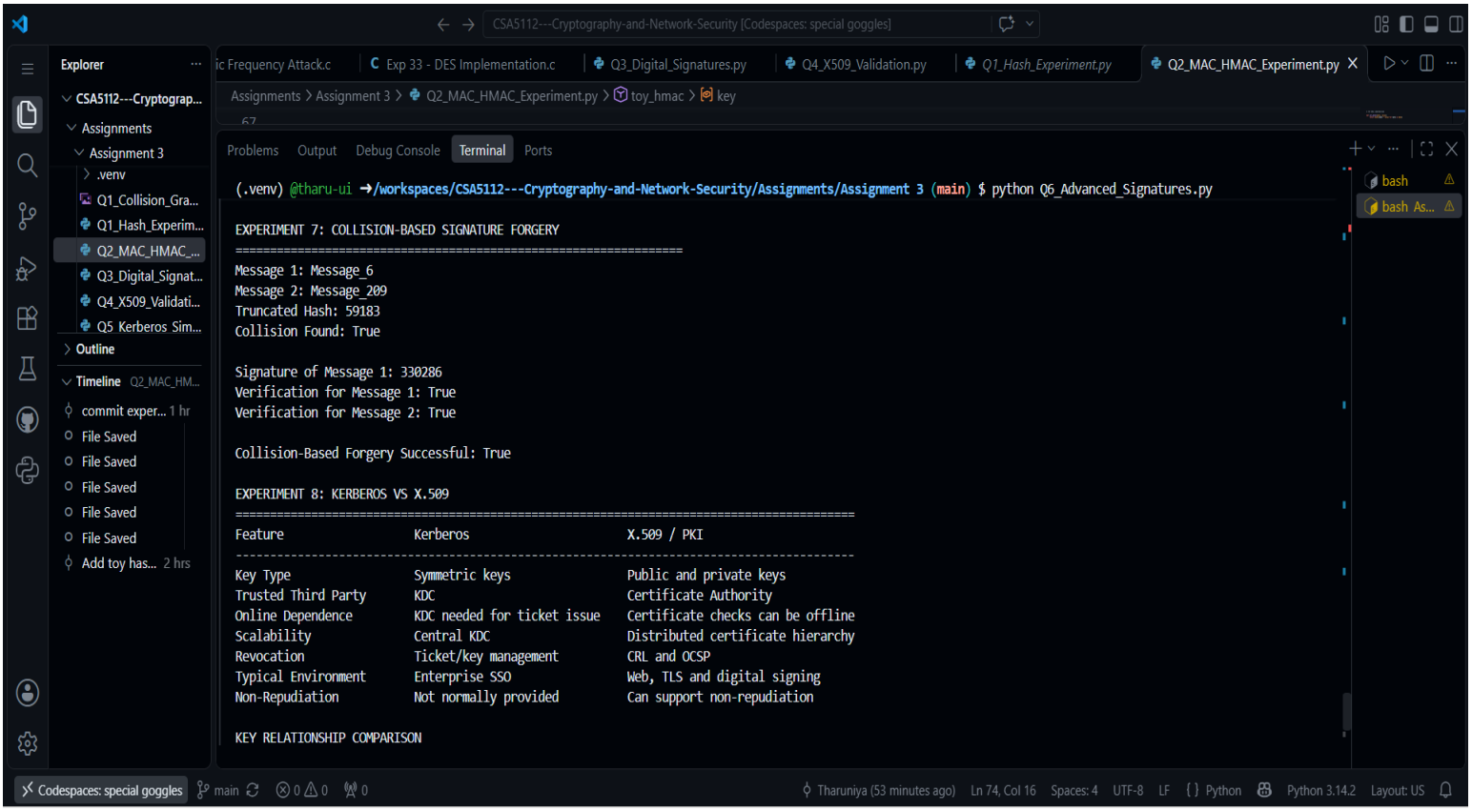
The collision-based forgery experiment showed that a weak hash can compromise digital signature security. The ElGamal nonce experiment demonstrated the risk of reusing signing randomness.

The comparison also showed that Kerberos is suitable for centralized authentication, whereas X.509/PKI is more appropriate when public verification and non-repudiation are required.




CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
CSA5112---Cryptograp...
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Validati...
Q5 Kerberos Sim...
Outline
Timeline Q2_MAC_HM...
commit exper... 1 hr
File Saved
File Saved
File Saved
File Saved
File Saved
Add toy hash, ...
ic Frequency Attack.c
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems Output Debug Console Terminal Ports
bash
bash As...
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q6_Advanced_Signatures.py
EXPERIMENT 3: ECDSA SIGNATURE
Curve: y^2 = x^3 + 2x + 2 mod 17
Generator: (5, 1)
Private Key: 7
Public Key: (0, 6)
Message: Elliptic Curve Digital Signature
Signature (r, s): (10, 2)
Valid Signature: True
Modified Message: False
EXPERIMENT 4: ELGAMAL SIGNATURE
Prime p: 467
Generator alpha: 2
Private Key: 127
Public Key: 132
Message: ElGamal Secure Message
Signature (r, s): (32, 171)
Valid Signature: True
Modified Message: False
Modified Signature: False
EXPERIMENT 5: SCHNORR SIGNATURE
Codespaces: special goggles
main
Tharuniya (53 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US


CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
CSA5112---Cryptograp...
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Validati...
Q5 Kerberos Sim...
Outline
Timeline Q2_MAC_HM...
commit exper... 1 hr
File Saved
File Saved
File Saved
File Saved
File Saved
Add toy has... 2 hrs
ic Frequency Attack.c
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems Output Debug Console Terminal Ports
bash
bash As...
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q6_Advanced_Signatures.py
Prime q: 11
Generator g: 2
Private Key: 3
Public Key: 8
Message: Schnorr Authentication Message
Signature (Commitment, Challenge, Response): (13, 6, 3)
Valid Signature: True
Modified Message: False
EXPERIMENT 6: ELGAMAL NONCE REUSE
Message 1: Payment Rs.1000
Message 2: Payment Rs.5000
Signature 1: (32, 391)
Signature 2: (32, 96)
Nonce Reused: 5
First r value: 32
Second r value: 32
Same r value: True
Observation: Reusing a signing nonce exposes
relationships that can enable private-key recovery.
A unique, unpredictable nonce is essential.
EXPERIMENT 7: COLLISION-BASED SIGNATURE FORGERY
Codespaces: special goggles
main
Tharuniya (53 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US


CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
CSA5112---Cryptograp...
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Validati...
Q5_Kerberos_Sim...
Outline
Timeline Q2_MAC_HM...
commit exper... 1 hr
File Saved
File Saved
File Saved
File Saved
File Saved
Add toy has... 2 hrs
ic Frequency Attack.c
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems
Output
Debug Console
Terminal
Ports
bash
bash As...
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q6_Advanced_Signatures.py
EXPERIMENT 7: COLLISION-BASED SIGNATURE FORGERY
Message 1: Message_6
Message 2: Message_209
Truncated Hash: 59183
Collision Found: True
Signature of Message 1: 330286
Verification for Message 1: True
Verification for Message 2: True
Collision-Based Forgery Successful: True
EXPERIMENT 8: KERBEROS VS X.509
Feature Kerberos X.509 / PKI
Key Type Symmetric keys Public and private keys
Trusted Third Party KDC Certificate Authority
Online Dependence KDC needed for ticket issue Certificate checks can be offline
Scalability Central KDC Distributed certificate hierarchy
Revocation Ticket/key management CRL and OCSP
Typical Environment Enterprise SSO Web, TLS and digital signing
Non-Repudiation Not normally provided Can support non-repudiation
KEY RELATIONSHIP COMPARISON
Codespaces: special goggles
main
Tharuniya (53 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US

Q7 – DESIGN A SECURE AUTHENTICATION AND INTEGRITY SCHEME FROM FIRST PRINCIPLES (10 MARKS)

Problem Context – Secure Over-the-Air Update for Medical Devices

A manufacturer distributes firmware updates to thousands of infusion pumps in hospitals. Each pump must be sure that an update (a) has not been altered, (b) really comes from the manufacturer, and (c) the manufacturer cannot later deny having issued it. Pumps have limited memory, are often offline, and have unreliable clocks. Some pumps must also authenticate to the hospital's central server before accepting a service session.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other:

Student A independently proposes:

- **Which security goals each mechanism will address**
- **Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where**
- **What key or certificate material each party holds**
- **A possible message flow for one update**

Student B independently proposes:

- **Which security goals each mechanism will address**
- **Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where**
- **What key or certificate material each party holds**
- **A possible message flow for one update**

Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent design thinking.

Stage 2 – Compare the Two Designs

The pair must now compare the two independent proposals. Discuss:

- **Where the designs agree?**
- **Where they differ?**
- **Strengths and weaknesses of each**
- **Missing threats or missing cases (rollback to an old version, stolen signing key, offline pump, wrong clock, revoked key)**
- **Which ideas should be retained?**

Evidence Expected: A short design-comparison record showing how the two initial ideas influenced the final solution.

Stage 3 – Joint Final Design

Using the strongest ideas from the discussion, jointly develop the final scheme. The final design must identify:

- **Threat model (who the attacker is and what the attacker can do) ● Security goals and the mechanism that achieves each**
- **Key and certificate management (issuance, storage, rotation, revocation) ● Message flow / protocol (diagram) for signing, distributing and verifying an update**
- **Behaviour when the pump is offline or its clock is wrong**
- **Complete pseudocode for signing and for verification**

Stage 4 – Divide the Validation Responsibilities

Student A – Security Argument Validation

Student A must:

- **State what each attack would need to succeed (e.g., 2^(n/2) work for a**

collision, forging a signature, compromising a key)

- **Show, goal by goal, why the final design prevents or limits each attack**
- **Estimate the attacker's effort for your chosen digest and key sizes using the results from Q1 and Q3**
- **Identify the residual risks that remain after the design**

Student B – Experimental Validation

Student B must:

- **Design correctness and attack test cases (valid update, modified image, modified signature, replayed older version, expired or revoked certificate, wrong key)**
- **Implement and instrument the final verification procedure**
- **Compare with a straightforward baseline (e.g., a bare SHA-256 digest published beside the file) to show which attacks the baseline fails to stop**
- **Record accept/reject outcomes, verification cost (hash operations, signature verifications) and the size of the added metadata**

Stage 5 – Cross-Defence

Student A must explain:

What experimental evidence supports the group's design?

Student B must explain:

How does the security argument follow from the group's choice of mechanisms, and what does each mechanism contribute?

This ensures that both students understand both the theoretical and experimental dimensions.

Stage 6 – Joint Evaluation

Together determine:

- **Which of integrity, authentication and non-repudiation is guaranteed by which part of the design**
- **What costs the design introduces (verification time, certificate storage, key-management burden)**
- **Under what conditions the design could still fail (key compromise, flawed implementation, compromised root, bad clock)**
- **Under what conditions a simpler or a different mechanism (e.g., HMAC with shared key, Kerberos tickets) would be more appropriate**

7.1 Problem Context

A manufacturer distributes firmware updates to thousands of medical infusion pumps installed in hospitals.

Each device must verify that:

1. The firmware has not been modified.
2. The firmware was genuinely issued by the manufacturer.
3. The manufacturer cannot easily deny having issued the signed update.
4. Older or unauthorized firmware cannot be installed.
5. The device can operate securely even when offline or when its clock is unreliable.

Some pumps must also authenticate to the hospital's central server before accepting a service session.

A secure Over-the-Air (OTA) update system must therefore combine cryptographic integrity, authentication, digital signatures, certificate management and rollback protection.

7.2 Stage 1 – Independent Design Proposal

Student A – Proposed Design

The proposed design uses SHA-256 for firmware integrity and a digital signature for manufacturer authentication.

The manufacturer maintains a private signing key, while each infusion pump stores the corresponding trusted public key or a trusted manufacturer certificate.

A signed firmware package contains the firmware, its version number, device model, hash and signature.

The pump verifies the signature and firmware digest before installation.

Student B – Proposed Design

The proposed design uses X.509 certificates to establish the manufacturer's identity and RSA digital signatures to authenticate firmware updates.

The pump stores a trusted root certificate and verifies the certificate chain before accepting the update.

A monotonic version counter is used to prevent rollback attacks.

The hospital service connection uses device certificates or a Kerberos-style authentication mechanism, depending on the hospital infrastructure.

The proposals above present the consolidated design direction; they should not be represented as separately recorded original submissions if no such records exist.

7.3 Stage 2 – Comparison of the Two Designs

Comparison Point	**Student A Proposal**	**Student B Proposal**
Integrity	SHA-256	SHA-256
Manufacturer Authentication	Digital signature	Digital signature and certificate
Trust Establishment	Pinned manufacturer public key	X.509 certificate chain
Rollback Protection	Firmware version checking	Monotonic version counter
Device Authentication	Trusted device identity	Device certificate or Kerberos
Main Advantage	Simple verification process	Stronger certificate-based identity management
Main Limitation	Requires secure public-key provisioning	Certificate management introduces additional complexity

Final Design Decision

The final design combines the strongest features of both proposals.

SHA-256 is used to calculate the firmware digest.

A digital signature authenticates the firmware package.

A trusted manufacturer certificate or pinned public key establishes the identity of the signing authority.

A protected monotonic version counter prevents installation of older firmware.

The device uses locally available trust information when offline, but offline operation must not silently bypass important revocation and certificate-security requirements.

7.4 Stage 3 – Threat Model

The system must consider the following possible attackers and threats.

Threat	**Attacker's Action**	**Security Risk**
Firmware Tampering	Modifies firmware before distribution	Malicious code may be installed
Fake Manufacturer	Creates an unauthorized firmware package	Unauthorized update may be accepted
Replay Attack	Reinstalls an older signed update	Vulnerable firmware may be restored
Signing-Key Theft	Steals the manufacturer's private key	Attacker may create apparently valid updates
Certificate Revocation	Attempts to use a revoked signing certificate	Compromised keys may remain trusted
Device Impersonation	Pretends to be a genuine infusion pump	Unauthorized service access
Clock Manipulation	Changes the device clock	Expired certificates may be accepted

Network Interception	Modifies or replaces update files	Firmware integrity may be compromised
Power Failure	Interrupts installation	Device may become unusable

The design must protect against these threats without depending entirely on continuous network connectivity.

7.5 Security Goals and Mechanisms

Security Goal	**Mechanism Used**	**Purpose**
Integrity	SHA-256	Detects firmware modification
Manufacturer Authentication	Digital Signature	Verifies the update source
Non-Repudiation	Manufacturer's protected private signing key, certificate and timestamp evidence	Provides evidence that the manufacturer issued the update
Rollback Protection	Monotonic version counter	Prevents installation of older firmware
Key Trust	X.509 certificate or pinned public key	Establishes the trusted signing identity
Revocation	Signed revocation information	Identifies compromised signing keys
Device Authentication	Device certificate or Kerberos-style tickets	Authenticates the pump to the hospital server
Availability	Offline verification and recovery process	Supports operation during network failures

7.6 Cryptographic Components

1. SHA-256

SHA-256 calculates a 256-bit digest of the firmware.

It is used to detect modifications to the firmware contents.

2. Digital Signature

The manufacturer signs the firmware digest and relevant update metadata using a protected private key.

The pump verifies the signature using the trusted public key.

For a production system, an approved signature scheme such as Ed25519 or ECDSA should be considered, subject to the device's cryptographic support and applicable regulatory requirements.

3. X.509 Certificate

An X.509 certificate associates the manufacturer's identity with the public signing key.

The device verifies the certificate chain or uses a securely provisioned trust anchor.

4. Monotonic Version Counter

The pump stores the highest accepted firmware version in protected non-volatile storage.

An update with a version number lower than or equal to the accepted version is rejected, except through a separately authorized recovery procedure.

5. Secure Boot

Secure boot verifies that the firmware being executed is trusted.

This prevents an attacker from bypassing update verification by replacing the installed firmware directly.

7.7 Key and Certificate Management

Manufacturer

The manufacturer generates and protects the firmware-signing private key.

The private key should be stored in a Hardware Security Module (HSM) or another suitably protected signing environment.

Infusion Pump

Each pump stores:

- Manufacturer's trusted public key or root certificate.
- Device identity and associated credentials.
- Highest accepted firmware version.
- Trusted revocation information.
- Secure boot verification information.

Key Rotation

The manufacturer periodically rotates signing keys.

New signing certificates must be introduced through a trusted update process.

The device should support a controlled transition between old and new trust anchors.

Key Revocation

If a manufacturer's private key is compromised, the corresponding certificate or signing key must be revoked.

The manufacturer should distribute signed revocation information through available trusted channels.

The device must reject updates signed by a revoked key once the applicable revocation policy establishes that the key is no longer trusted.

7.8 Stage 3 – Message Flow for a Secure Firmware Update

Secure OTA Update Diagram

Step 1 – Firmware Preparation

Manufacturer prepares the firmware.

↓

Step 2 – Hash Calculation

Manufacturer calculates the SHA-256 digest.

↓

Step 3 – Digital Signature

Manufacturer signs the digest and update metadata using its private key.

↓

Step 4 – Distribution

Signed firmware package is distributed to the infusion pump.

↓

Step 5 – Device Verification

Pump checks the device model, version, signing certificate and signature.

↓

Step 6 – Integrity Verification

Pump calculates the firmware digest and compares it with the signed digest.

↓

Step 7 – Rollback Check

Pump compares the update version with its protected version counter.

↓

Step 8 – Installation

If all checks succeed, the pump installs the update using a safe update procedure.

↓

Step 9 – Secure Boot

The pump verifies the installed firmware before executing it.

7.9 Signed Update Package Structure

The update package should contain:

Field	**Description**
Device Model	Identifies the compatible pump model

Firmware Version	Identifies the release version
Firmware Size	Indicates the expected firmware size
SHA-256 Digest	Digest of the firmware contents
Signing Key Identifier	Identifies the trusted signing key
Certificate Information	Establishes the manufacturer's identity
Signature	Authenticates the digest and metadata
Release Information	Identifies the update release and security requirements

The signature must cover the relevant metadata as well as the firmware digest. Otherwise, an attacker may attempt to change the model, version or other security-critical information independently of the firmware contents.

7.10 Pseudocode for Firmware Signing

```
FUNCTION SignFirmware(firmware, device_model, version):

    digest = SHA256(firmware)

    metadata = {
        device_model,
        version,
        firmware_size,
        digest,
        signing_key_id
    }

    signed_data = Encode(metadata)

    signature = Sign(
        manufacturer_private_key,
        signed_data
    )
```

```
    package = {
        firmware,
        metadata,
        signature,
        manufacturer_certificate
    }

    RETURN package
```

7.11 Pseudocode for Firmware Verification

```
FUNCTION VerifyFirmware(package, device):

    IF package.device_model != device.model:
        REJECT

    IF package.version <= device.highest_version:
        REJECT

    IF signing_key_is_revoked(package.signing_key_id):
        REJECT

    IF certificate_is_not_trusted(package.certificate):
        REJECT

    IF certificate_policy_is_not_satisfied:
        REJECT

    signed_data = Encode(package.metadata)

    IF VerifySignature(
        trusted_public_key,
```

```
    signed_data,
    package.signature
) == FALSE:
    REJECT

calculated_hash = SHA256(package.firmware)

IF calculated_hash != package.metadata.digest:
    REJECT

IF package.firmware_size != actual_size(package.firmware):
    REJECT

INSTALL firmware using safe update procedure

UPDATE protected version counter

VERIFY installed firmware using secure boot

ACCEPT
```

7.12 Offline Operation and Unreliable Clocks

The infusion pumps may not always have access to the internet or a reliable clock.

Therefore, certificate verification must not depend exclusively on obtaining a fresh online response.

A suitable offline policy includes:

1. Store the trusted manufacturer public key or root certificate securely.
2. Store signed revocation information received during a previous trusted connection.
3. Record the issue time and expiry time of revocation information.

4. Use a protected monotonic counter or trusted time source where available.
5. Apply a defined offline grace period for cached revocation information.
6. Reject high-risk updates if trust or revocation information is too stale.
7. Never treat an unavailable clock as automatic proof that a certificate is valid.

A device must not silently accept an expired or revoked certificate merely because it is offline.

The final policy should balance patient safety, availability and cryptographic assurance.

7.13 Hospital Server Authentication

Some pumps must authenticate to a hospital's central server before accepting a service session.

This can be implemented using device certificates and mutual TLS or a Kerberos-based authentication framework.

A device certificate allows the hospital server to verify the identity of the pump.

The hospital server can also authenticate itself to the pump.

A Kerberos-style ticket system may be suitable when the hospital already operates a trusted KDC and centralized identity infrastructure.

For medical-device service sessions, the authentication mechanism should also enforce authorization rules, session time limits and audit logging.

7.14 Stage 4 – Experimental Validation

A Python program named Q7_Secure_OTA_Update.py was implemented.

The experiment simulated the verification of a firmware package before installation.

Firmware Details

Parameter	**Result**
Device Model	INFUSION-PUMP-X1

Installed Version	1
New Version	2
SHA-256	ebc0bc554c5178fe8a861f903db7294ce235c4c612b94967574f57cd8983374e
Signing Key	MANUFACTURER-KEY-01
Added Metadata Size	254 bytes

7.15 Experimental Results – OTA Security Tests

Test No.	**Test Case**	**Result**	**Reason**
1	Valid Firmware Update	ACCEPTED	Update accepted
2	Modified Firmware	REJECTED	Firmware integrity check failed
3	Modified Signature	REJECTED	Digital signature verification failed
4	Replayed Older Version	REJECTED	Rollback or duplicate version rejected
5	Revoked Signing Key	REJECTED	Signing key has been revoked
6	Signature Created with Wrong Private Key	REJECTED	Digital signature verification failed
7	Expired Certificate	REJECTED	Certificate has expired
8	Wrong Device Model	REJECTED	Device model mismatch
9	Offline Device – Clock Unavailable	ACCEPTED	Update accepted

7.16 Verification Cost

The program recorded the number of hash operations and signature verifications performed for each test.

Test Case	Hash Operations	Signature Verifications
Valid Firmware Update	1	1
Modified Firmware	1	0
Modified Signature	1	1
Replayed Older Version	0	0
Revoked Signing Key	0	0
Wrong Private Key	1	1
Expired Certificate	0	0
Wrong Device Model	0	0
Offline Device	1	1

The system performs inexpensive preliminary checks before cryptographic verification where possible.

For example, an update with the wrong device model or an unacceptable version can be rejected without calculating the firmware hash or verifying the signature.

7.17 Baseline Comparison – Unsigned Hash Attack

A simple baseline system publishes the firmware and its SHA-256 digest without digitally signing the digest.

An attacker who can replace both the firmware and its published digest can cause the receiver to accept the modified firmware.

Experimental Results

Parameter	Result

Original Firmware Hash	ebc0bc554c5178fe8a861f903db7294ce235c4c612b94967574f57cd8983374e
Attacker's Published Hash	3275d3b4beb91098a5c2403fa8f54b4337ef643ba1454a7369ee595b24f3e3f5
Baseline Accepts Modified Firmware	True

Observation

The baseline accepted the modified firmware because the attacker replaced both the firmware and the unsigned hash.

This demonstrates that a plain hash does not authenticate the manufacturer.

The digital-signature-based design prevents this attack because the attacker cannot generate a valid signature without the manufacturer's private key.

7.18 Analysis of Attack Resistance

Attack	**Protection Mechanism**	**Result**
Firmware Modification	SHA-256 and digital signature	Detected
Fake Firmware Publisher	Manufacturer's digital signature	Rejected
Replayed Old Firmware	Monotonic version counter	Rejected
Stolen or Revoked Signing Key	Revocation policy	Rejected after revocation is securely established
Wrong Signing Key	Public-key signature verification	Rejected

Wrong Device Model	Signed model metadata and device check	Rejected
Certificate Expiry	Certificate validity policy	Rejected when validity can be established
Network Interception	Signature and integrity verification	Modification detected
Power Failure	Atomic update and recovery design	Must be handled by production implementation

7.19 Stage 5 – Cross-Defence

Student A: What Experimental Evidence Supports the Design?

The experiment demonstrated that valid firmware was accepted, whereas modified firmware and signatures were rejected.

Older firmware, revoked signing keys, expired certificates and incorrect device models were also rejected.

The baseline test demonstrated that an unsigned hash could be replaced by an attacker.

Therefore, the experimental results support the use of digital signatures, SHA-256, trusted keys and rollback protection.

Student B: How Does the Security Argument Follow from the Design?

SHA-256 provides a digest for detecting firmware changes.

Digital signatures authenticate the update and connect it to the manufacturer's private key.

X.509 certificates or pinned public keys establish the trust relationship.

Monotonic version counters prevent rollback.

Revocation information helps invalidate compromised signing keys.

Device authentication protects service sessions between pumps and the hospital server.

Each mechanism addresses a different security requirement, and the combination provides stronger protection than using any one mechanism alone.

7.20 Stage 6 – Joint Evaluation

Security Guarantees

Security Property	Mechanism	Explanation
Integrity	SHA-256	Detects changes to firmware contents
Authentication	Digital Signature	Verifies that the update was signed using the trusted private key
Non-Repudiation	Protected private key, certificate and timestamp evidence	Provides evidence of the manufacturer's authorization
Rollback Protection	Monotonic Version Counter	Prevents installation of older firmware
Trust Establishment	X.509 or Pinned Public Key	Establishes the trusted manufacturer identity
Device Authentication	Device Certificate or Kerberos	Authenticates the device to the hospital server

Costs Introduced by the Design

1. Verification Time

The pump must calculate a firmware hash and verify a digital signature before installation.

2. Certificate Storage

Trusted certificates and revocation information require additional storage.

3. Key Management

The manufacturer must protect, rotate and revoke signing keys.

4. Implementation Complexity

The update process must handle interrupted installations, version management, secure boot and recovery.

5. Network Requirements

Revocation information and certificate updates may need to be distributed periodically.

7.21 Residual Risks and Production Improvements

Although the proposed design provides strong security, some risks remain.

Private-Key Compromise

If the manufacturer's signing key is stolen, an attacker may create valid-looking firmware updates. The key must be protected using an HSM, restricted signing permissions and a documented emergency revocation process.

Compromised Root Certificate

If the trusted root CA is compromised, certificate trust may be undermined. Trust-anchor rotation and secure device provisioning are required.

Faulty Implementation

Incorrect signature verification, unsafe parsing or insecure firmware installation may introduce vulnerabilities even when the cryptographic algorithms are strong.

Rollback Counter Manipulation

If an attacker can modify the stored version counter, older firmware may be installed. The counter must be protected using secure hardware or equivalent tamper-resistant storage.

Power Failure During Installation

A partially installed firmware image may make the device unusable. A production system should use an atomic update process, such as an A/B firmware partition, with recovery to a known-good image.

Offline Revocation Limitations

A pump cannot know about a newly revoked certificate until it receives updated revocation information. The system must define a safe offline policy rather than automatically accepting updates when revocation checks are unavailable.

Medical Safety

A cryptographically authentic update may still contain software defects. Firmware must also undergo testing, validation, safety assessment and controlled deployment before installation on medical devices.

7.22 Why Would HMAC or Kerberos Alone Be Insufficient?

HMAC is suitable when the manufacturer and receiver share a secret key. However, a pump containing a shared manufacturer authentication secret may create a risk if the secret is extracted from one compromised device.

HMAC also does not provide public verifiability or non-repudiation.

Kerberos is suitable for hospital service authentication but depends on a reachable and trusted KDC for ticket issuance.

Neither mechanism alone is the ideal choice for independently verifying manufacturer-issued firmware updates on offline devices.

A digital signature allows each pump to verify the update using a public key without possessing the manufacturer's private signing key.

7.23 Conclusion

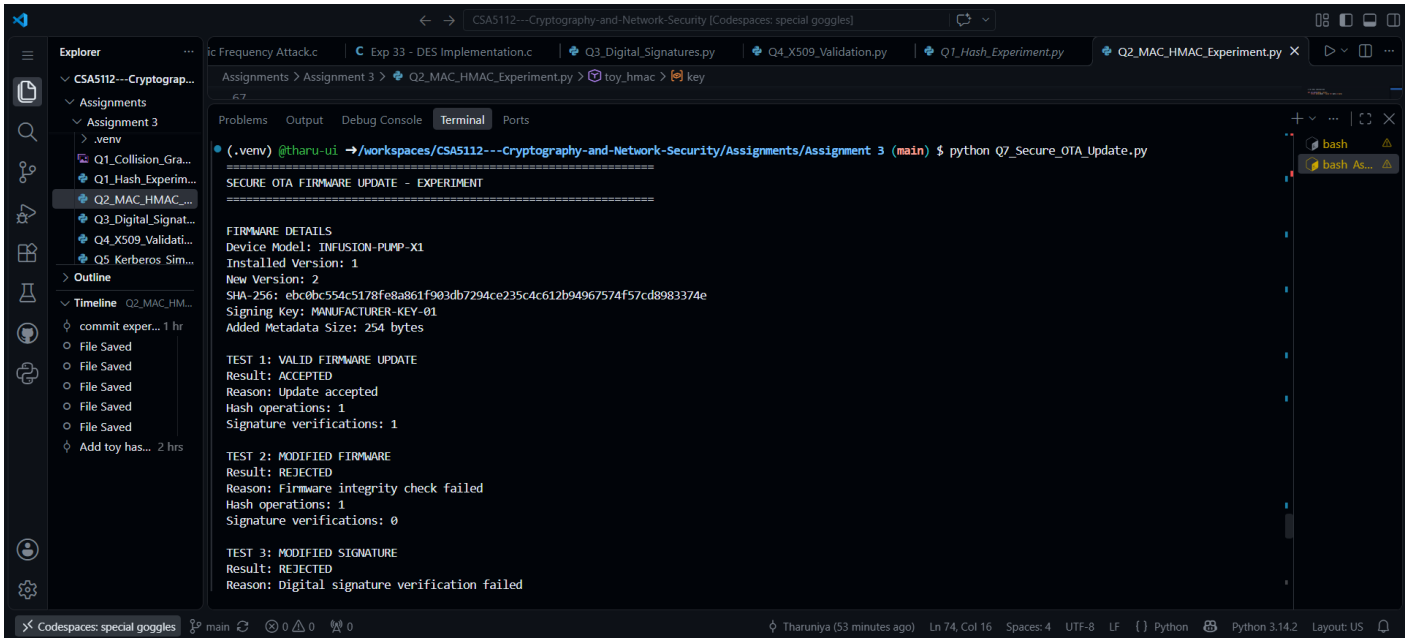
The proposed secure OTA update system combines SHA-256, digital signatures, trusted certificates, rollback protection and device authentication.

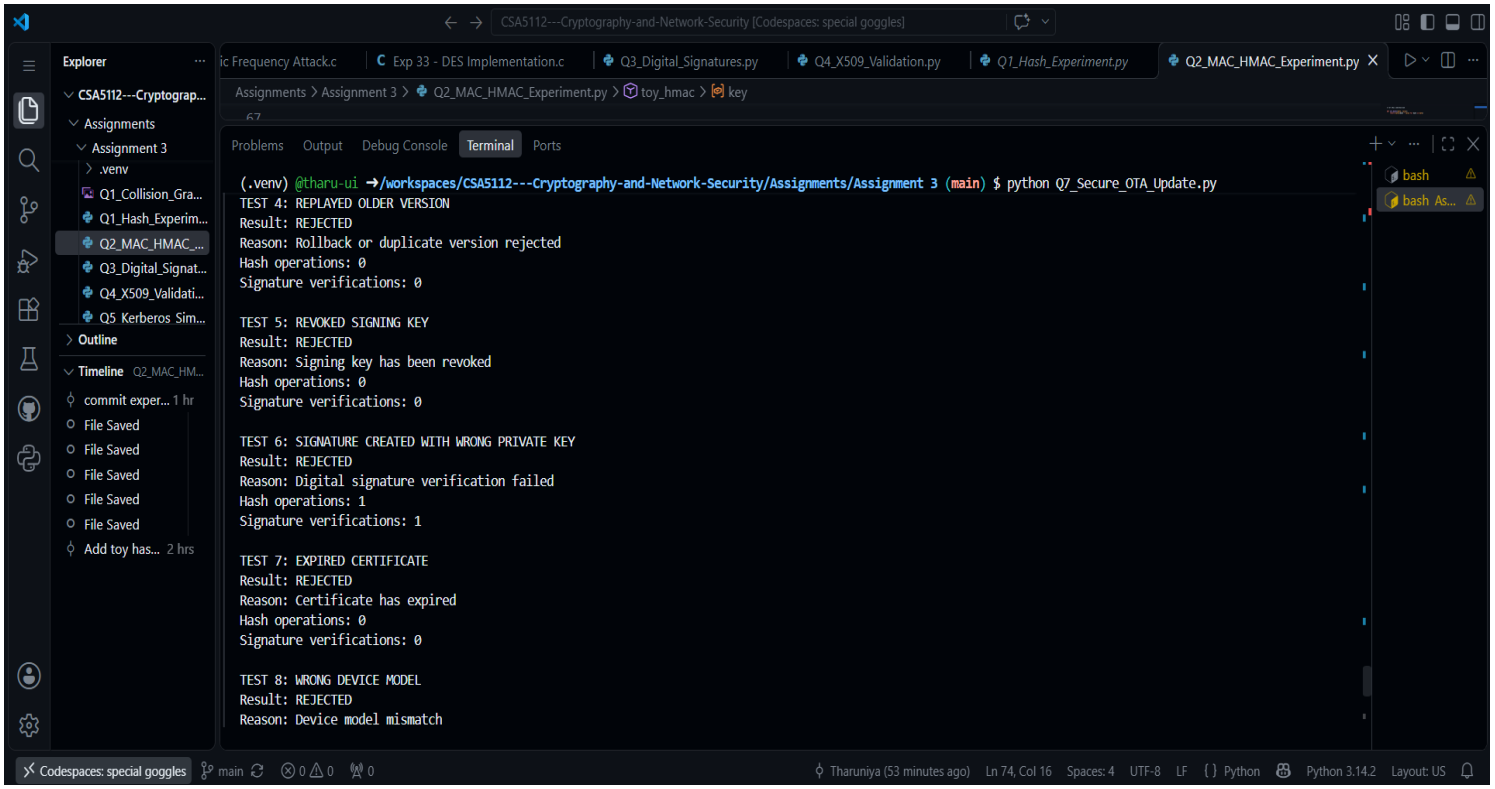
The experimental results showed that valid firmware was accepted, while modified firmware, modified signatures, older versions, revoked keys and incorrect device models were rejected.

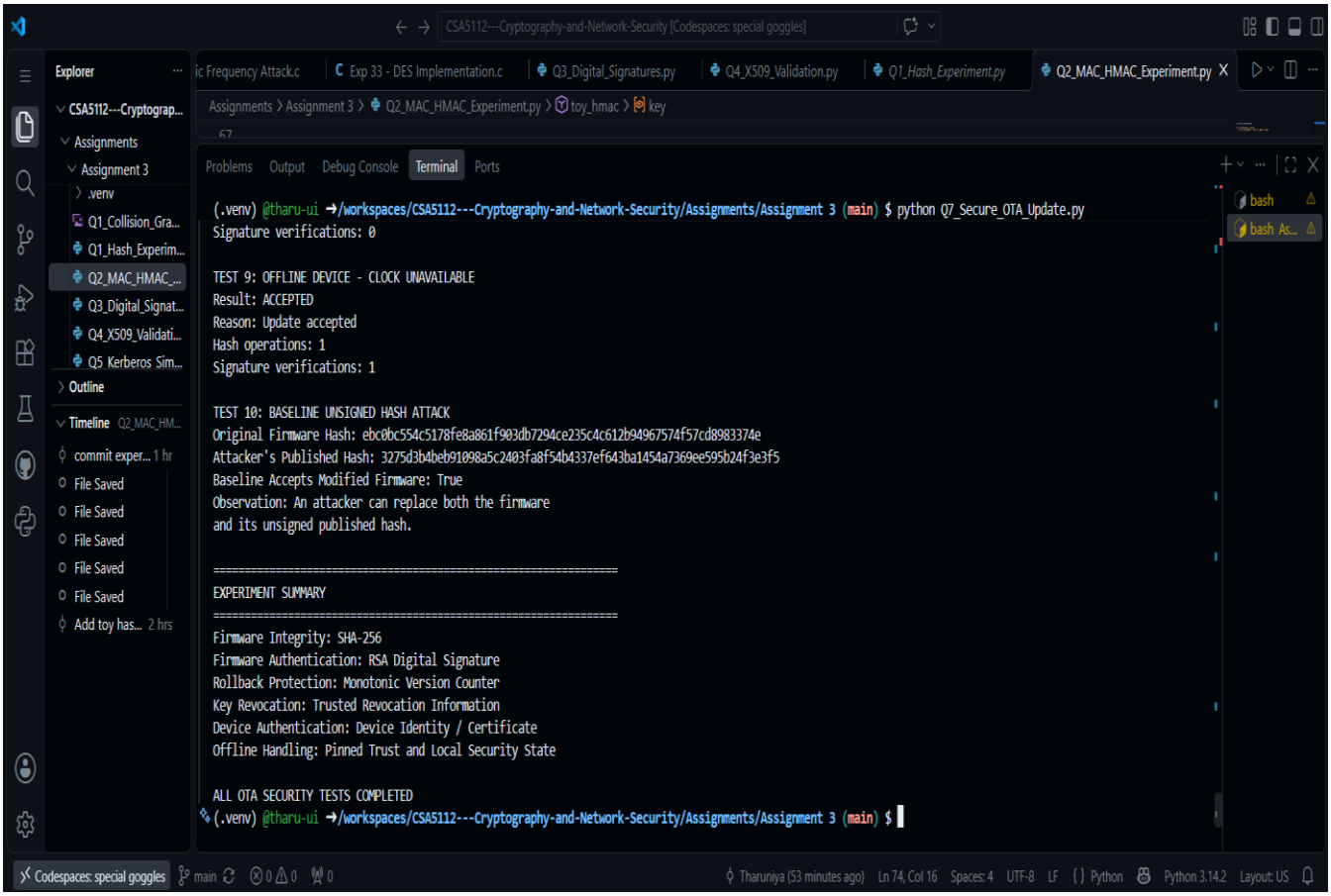
The baseline experiment demonstrated that an unsigned hash can be replaced by an attacker.

The final design is therefore more secure because it verifies both firmware integrity and manufacturer authenticity.

For real medical devices, the design must also include secure boot, protected version counters, atomic installation, recovery mechanisms, careful offline revocation policies and appropriate safety validation.


CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems Output Debug Console Terminal Ports
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q7_Secure_OTA_Update.py
SECURE OTA FIRMWARE UPDATE - EXPERIMENT
FIRMWARE DETAILS
Device Model: INFUSION-PUMP-X1
Installed Version: 1
New Version: 2
SHA-256: ebc0bc554c5178fe8a861f903db7294ce235c4c612b94967574f57cd8983374e
Signing Key: MANUFACTURER-KEY-01
Added Metadata Size: 254 bytes
TEST 1: VALID FIRMWARE UPDATE
Result: ACCEPTED
Reason: Update accepted
Hash operations: 1
Signature verifications: 1
TEST 2: MODIFIED FIRMWARE
Result: REJECTED
Reason: Firmware integrity check failed
Hash operations: 1
Signature verifications: 0
TEST 3: MODIFIED SIGNATURE
Result: REJECTED
Reason: Digital signature verification failed


CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems Output Debug Console Terminal Ports
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q7_Secure_OTA_Update.py
TEST 4: REPLAYED OLDER VERSION
Result: REJECTED
Reason: Rollback or duplicate version rejected
Hash operations: 0
Signature verifications: 0
TEST 5: REVOKED SIGNING KEY
Result: REJECTED
Reason: Signing key has been revoked
Hash operations: 0
Signature verifications: 0
TEST 6: SIGNATURE CREATED WITH WRONG PRIVATE KEY
Result: REJECTED
Reason: Digital signature verification failed
Hash operations: 1
Signature verifications: 1
TEST 7: EXPIRED CERTIFICATE
Result: REJECTED
Reason: Certificate has expired
Hash operations: 0
Signature verifications: 0
TEST 8: WRONG DEVICE MODEL
Result: REJECTED
Reason: Device model mismatch


CSA5112---Cryptography-and-Network-Security [Codespaces: special goggles]
Explorer
CSA5112---Cryptograp...
Assignments
Assignment 3
.venv
Q1_Collision_Gra...
Q1_Hash_Experim...
Q2_MAC_HMAC_...
Q3_Digital_Signat...
Q4_X509_Validati...
Q5_Kerberos_Sim...
Outline
Timeline Q2_MAC_HM...
commit exper... 1 hr
File Saved
File Saved
File Saved
File Saved
File Saved
Add toy has... 2 hrs
ic Frequency Attack.c
Exp 33 - DES Implementation.c
Q3_Digital_Signatures.py
Q4_X509_Validation.py
Q1_Hash_Experiment.py
Q2_MAC_HMAC_Experiment.py
Assignments > Assignment 3 > Q2_MAC_HMAC_Experiment.py > toy_hmac > key
Problems
Output
Debug Console
Terminal
Ports
bash
bash As...
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $ python Q7_Secure_OTA_Update.py
Signature verifications: 0
TEST 9: OFFLINE DEVICE - CLOCK UNAVAILABLE
Result: ACCEPTED
Reason: Update accepted
Hash operations: 1
Signature verifications: 1
TEST 10: BASELINE UNSIGNED HASH ATTACK
Original Firmware Hash: ebc0bc554c5178fe8a861f903db7294ce235c4c612b94967574f57cd8983374e
Attacker's Published Hash: 3275d3b4beb91098a5c2403fa8f54b4337ef643ba1454a7369ee595b24f3e3f5
Baseline Accepts Modified Firmware: True
Observation: An attacker can replace both the firmware
and its unsigned published hash.
EXPERIMENT SUMMARY
Firmware Integrity: SHA-256
Firmware Authentication: RSA Digital Signature
Rollback Protection: Monotonic Version Counter
Key Revocation: Trusted Revocation Information
Device Authentication: Device Identity / Certificate
Offline Handling: Pinned Trust and Local Security State
ALL OTA SECURITY TESTS COMPLETED
(.venv) @tharu-ui ➜/workspaces/CSA5112---Cryptography-and-Network-Security/Assignments/Assignment 3 (main) $
Codespaces: special goggles
main
Tharuniya (53 minutes ago)
Ln 74, Col 16
Spaces: 4
UTF-8
LF
Python
Python 3.14.2
Layout: US